Reasoning language models & Test-time scaling

CS 5624: Natural Language Processing

Fall 2026

https://tuvllms.github.io/nlp-fall-2026/

Tu Vu

VIRGINIA TECH

Logistics

- Quiz2, HW2 coming
- Proposal feedback EOW

AI news

Students can register here: https://forms.gle/5muQBbhwmPowepzx8



Post-training review

SFT

The tokens come from a **human/reference response**:

$$o = (o_1, \ldots, o_T) \sim \text{human data}.$$

We maximize

$$\boxed{J_{\text{SFT}}(\theta) = \frac{1}{T} \sum_{t=1}^{T} \log \pi_\theta(o_t \mid o_{<t})}$$

So the model is told:

"Make the human's tokens more likely."

RL

The tokens are **sampled from the model**:

$$o = (o_1, \ldots, o_T) \sim \pi_{\theta_{\text{old}}}(\cdot \mid x)$$

Then we evaluate the generated response, get reward r, and derive advantage A.

Conceptually, the objective is

$$\boxed{J_{\text{RL}}(\theta) = \frac{1}{T} \sum_{t=1}^{T} A \log \pi_\theta(o_t \mid x, o_{<t})}$$

Thus:

$$A > 0 \quad \Rightarrow \quad \text{make the model's sampled tokens more likely}$$

and

$$A < 0 \quad \Rightarrow \quad \text{make the model's sampled tokens less likely.}$$

Comparison of Policy Gradient Methods

Method	Advantage definition	Token-level advantage?	Value model
REINFORCE	$A = R - b$	❌ No	❌ No
RLOO	$A^{(k)} = R^{(k)} - \frac{1}{K-1}\sum_{i \neq k} R^{(i)}$	❌ No	❌ No
PPO	A_t from TD / GAE	✅ Yes	✅ Yes
GRPO	$A^{(k)} = \frac{R^{(k)} - \mu}{\sigma}$	❌ No	❌ No

Comparison of Policy Gradient Methods

Method	Policy objective per token L_t	KL usage	Key idea
REINFORCE	$L_t = \log \pi_\theta(a_t \mid s_t)\, A$	Often used in RLHF as a reward penalty to a reference policy: $r_t \leftarrow r_t - \beta\, \mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$	Reinforce tokens of high-reward sequences
RLOO	$L_t = \log \pi_\theta(a_t \mid s_t)\, A^{(k)}$	Often used in RLHF as a reward penalty to a reference policy: $r_t \leftarrow r_t - \beta\, \mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$	Compare each completion with others
PPO	$L_t = \min(\rho_t A_t,\ \mathrm{clip}(\rho_t, 1-\epsilon, 1+\epsilon) A_t)$	**Implicit constraint via clipping** Clipping on $\rho_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}$ keeps updates close to the rollout policy π_{old}. **Often additionally used in RLHF** as a reward penalty to a reference policy: $r_t \leftarrow r_t - \beta\, \mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$	Stable updates + token-level credit
GRPO	$L_t = \min(\rho_t A^{(k)},\ \mathrm{clip}(\rho_t, 1-\epsilon, 1+\epsilon) A^{(k)})$	**Implicit constraint via clipping** Clipping on $\rho_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}$ keeps updates close to the rollout policy π_{old}. **Often additionally used in RLHF** as a reward penalty to a reference policy: $r_t \leftarrow r_t - \beta\, \mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$	PPO-style update without value model

GRPO

$$J(\theta) = \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|a_i|}\sum_{t=1}^{|a_i|}\left(\min\left(\frac{\pi_\theta(a_{i,t}|s_i)}{\pi_{\theta_{\text{old}}}(a_{i,t}|s_i)}A_{i,t},\ \text{clip}\left(\frac{\pi_\theta(a_{i,t}|s_i)}{\pi_{\theta_{\text{old}}}(a_{i,t}|s_i)}, 1-\varepsilon, 1+\varepsilon\right)A_{i,t}\right)\right.$$
$$\left. - \beta\mathcal{D}_{\text{KL}}(\pi_\theta(\cdot|s_i)\|\pi_{\text{ref}}(\cdot|s_i))\right)$$

With the advantage computation for the completion index i:

$$A_i = \frac{r_i - \text{mean}(r_1, r_2, \cdots, r_G)}{\text{std}(r_1, r_2, \cdots, r_G)}.$$

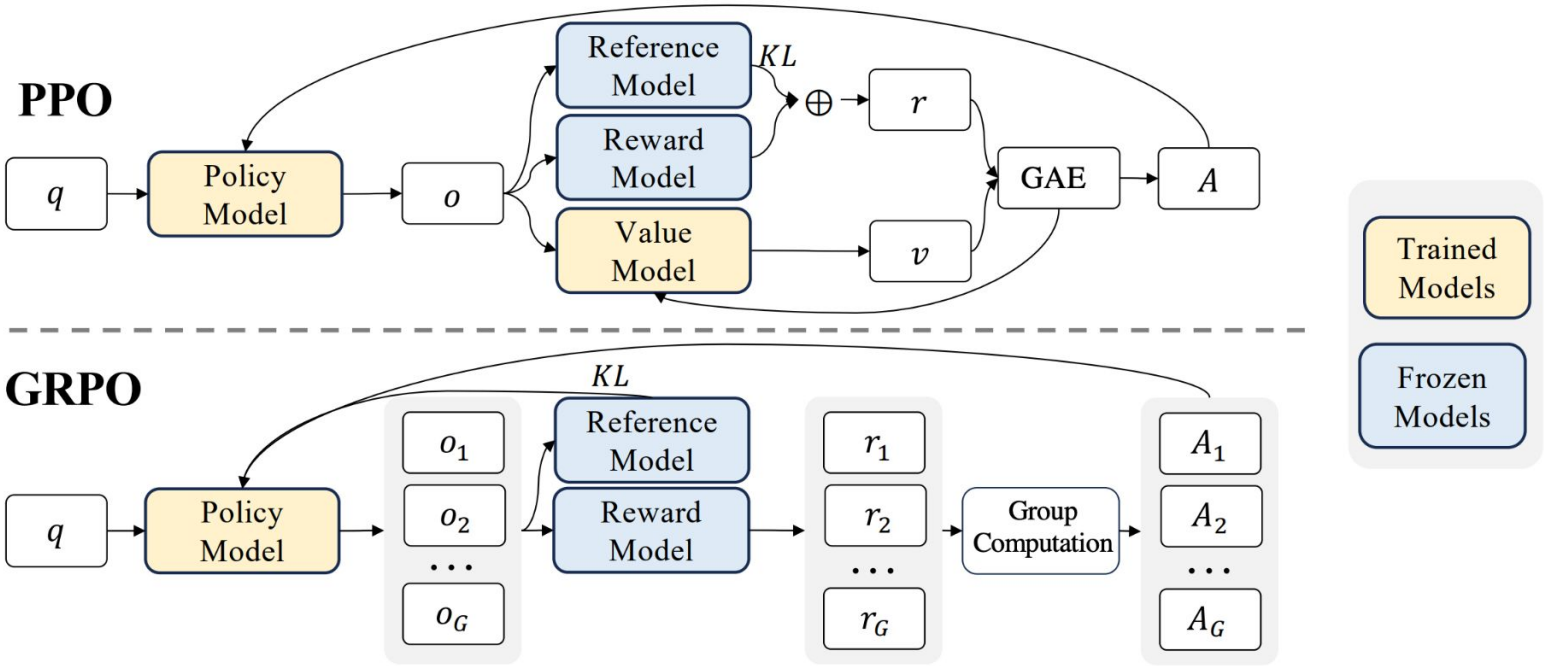


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

DPO vs. RLHF

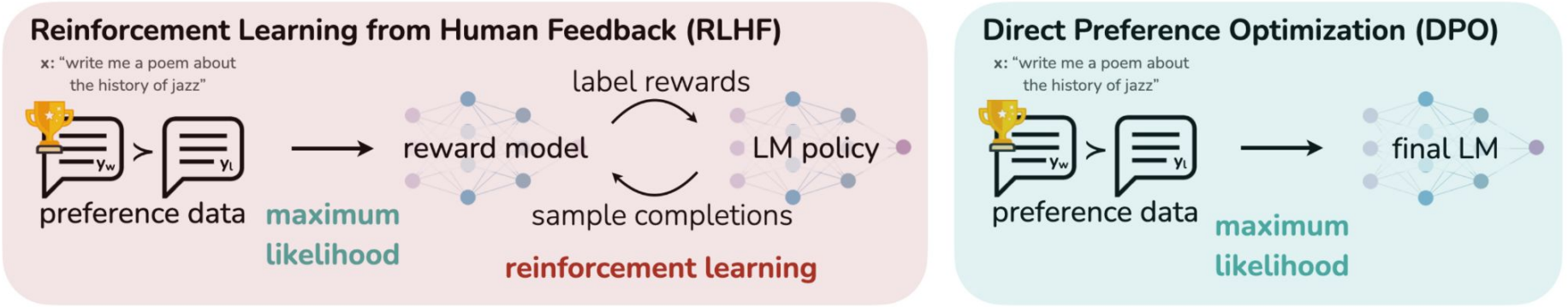


Figure 1: **DPO optimizes for human preferences while avoiding reinforcement learning.** Existing methods for fine-tuning language models with human feedback first fit a reward model to a dataset of prompts and human preferences over pairs of responses, and then use RL to find a policy that maximizes the learned reward. In contrast, DPO directly optimizes for the policy best satisfying the preferences with a simple classification objective, fitting an *implicit* reward model whose corresponding optimal policy can be extracted in closed form.

DPO objective

$$L_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim D}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

DPO says: deviate from the reference model enough to better match the preferences, but use the reference model as an anchor.

RL*F

RLAIF

Constitutional AI: Harmlessness from AI Feedback

Yuntao Bai,* Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion,

Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon,
Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain,
Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller,
Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosuite, Liane Lovitt,
Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemi Mercado, Nova DasSarma,
Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec,
Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly,
Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann,
Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, Jared Kaplan*

Anthropic

RLCF

Checklists Are Better Than Reward Models For Aligning Language Models

Vijay Viswanathan ♡ **Yanchao Sun**♣ **Shuang Ma**♣* **Xiang Kong**♣
Meng Cao♣ **Graham Neubig**♡ **Tongshuang Wu**♡
♡ Carnegie Mellon University ♣Apple

Abstract

Language models must be adapted to understand and follow user instructions. Reinforcement learning is widely used to facilitate this – typically using fixed criteria such as "helpfulness" and "harmfulness". In our work, we instead propose using flexible, instruction-specific criteria as a means of broadening the impact that reinforcement learning can have in eliciting instruction following. We propose "**Reinforcement Learning from Checklist Feedback**" (**RLCF**). From instructions, we extract checklists and evaluate how well responses satisfy each item—using both AI judges and specialized verifier programs—then combine these scores to compute rewards for RL. We compare RLCF with other alignment methods on top of a strong instruction following model (`Qwen2.5-7B-Instruct`) on five widely-studied benchmarks – **RLCF is the only method to help on every benchmark**, including a 4-point boost in hard satisfaction rate on FollowBench, a 6-point increase on InFoBench, and a 3-point rise in win rate on Arena-Hard. We show that RLCF can also be used off-policy to improve `Llama 3.1 8B Instruct` and `OLMo 2 7B Instruct`. These results establish rubrics as a key tool for improving language models' support of queries that express a multitude of needs. We release our our dataset of rubrics (*WildChecklists*), models, and code to the public.[1]

Rubrics-based

scale

Rubrics as Rewards: Reinforcement Learning Beyond Verifiable Domains

Anisha Gunjal, Anthony Wang*, Elaine Lau, Vaskar Nath, Yunzhong He, Bing Liu, and Sean Hendryx

Scale AI

**Work done during internship at Scale AI*

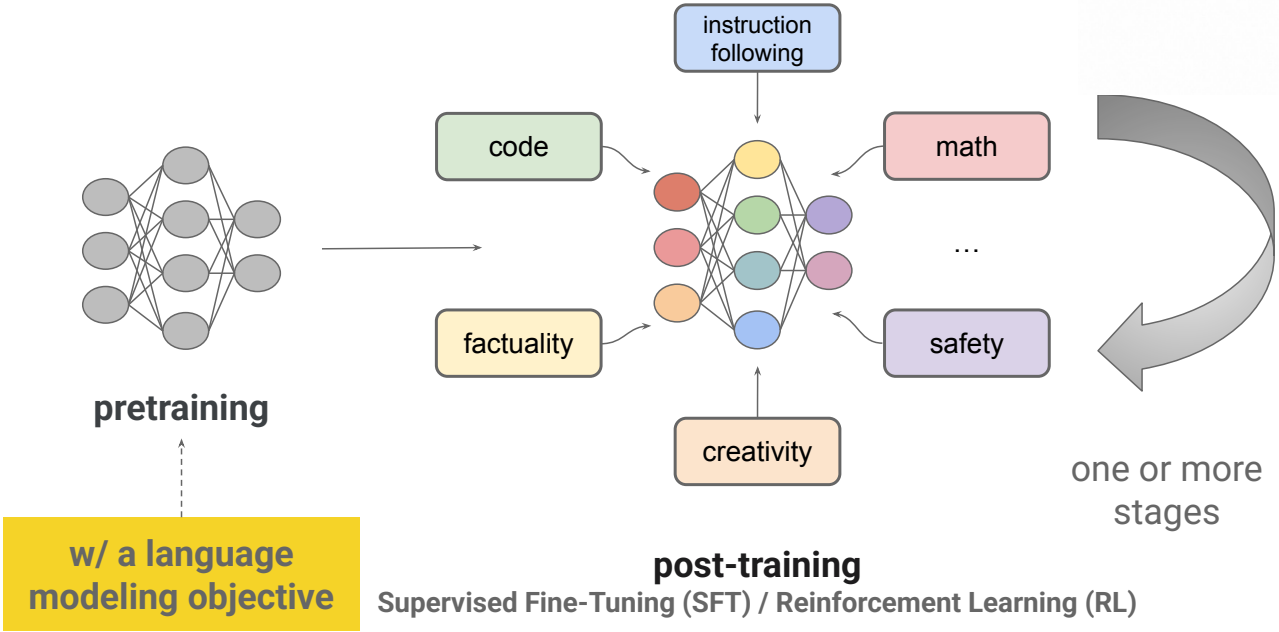
anisha.gunjal@scale.com

6v2 [cs.LG] 3 Oct 2025

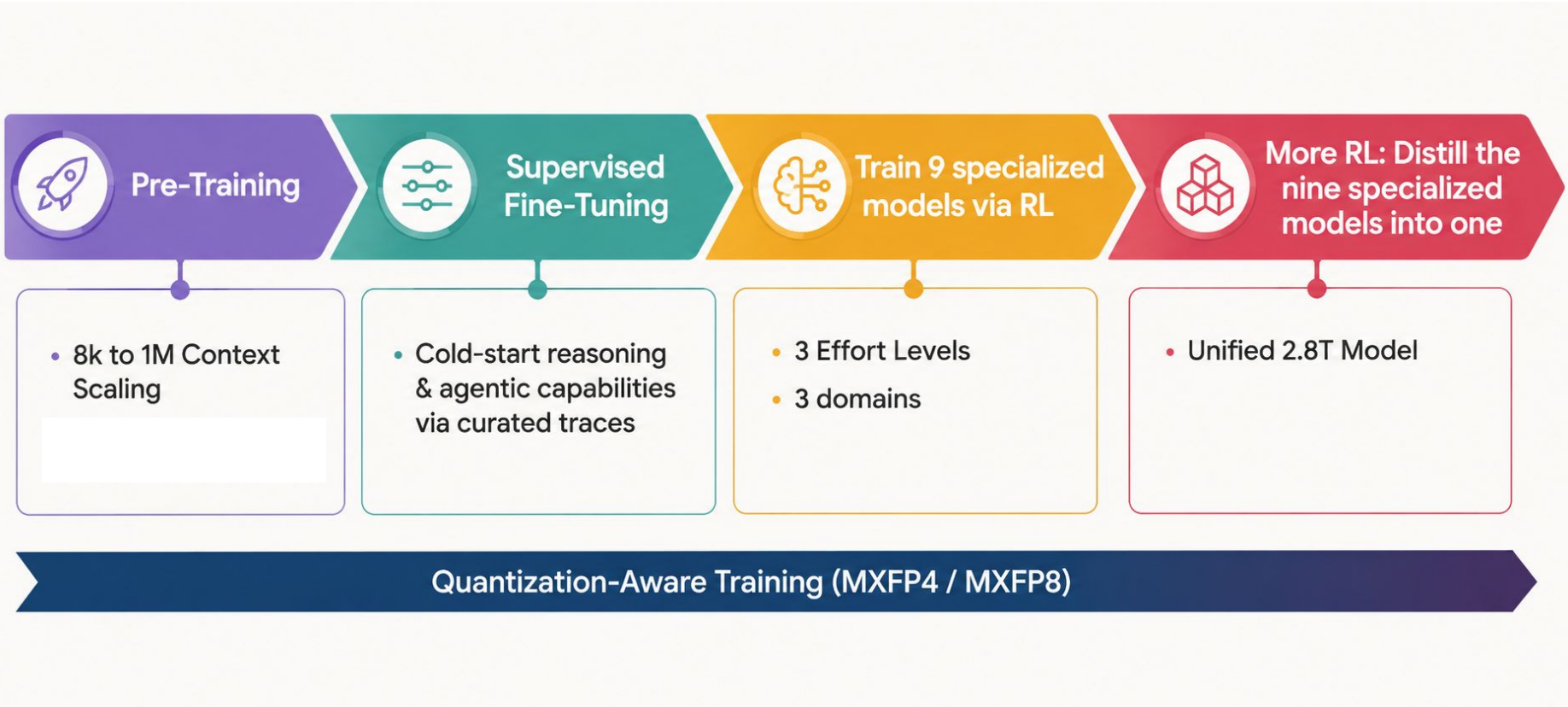
Abstract

Reinforcement Learning with Verifiable Rewards (RLVR) has proven effective for complex reasoning tasks with clear correctness signals such as math and coding. However, extending it to real-world reasoning tasks is challenging, as evaluation depends on nuanced, multi-criteria judgments rather than binary correctness. Instance-specific rubrics have recently been used in evaluation benchmarks to capture such judgments, but their potential as reward signals for on-policy post-training remains underexplored. We introduce **Rubrics as Rewards (*RaR*)**, an on-policy reinforcement learning method that extends RLVR beyond verifiable domains by using rubric-based feedback. Across both medical and science domains, we evaluate multiple strategies for aggregating rubric feedback into rewards. The best RaR variant achieves relative improvements of up to 31% on HealthBench and 7% on GPQA-Diamond over popular LLM-as-judge baselines that rely on direct Likert-based rewards. These results demonstrate that RaR-trained policies adapt well to diverse evaluation formats, performing strongly on both rubric-based and multiple-choice tasks. Moreover, we find that using rubrics as structured reward signals yields better alignment for smaller judges and reduces performance variance across judge scales.

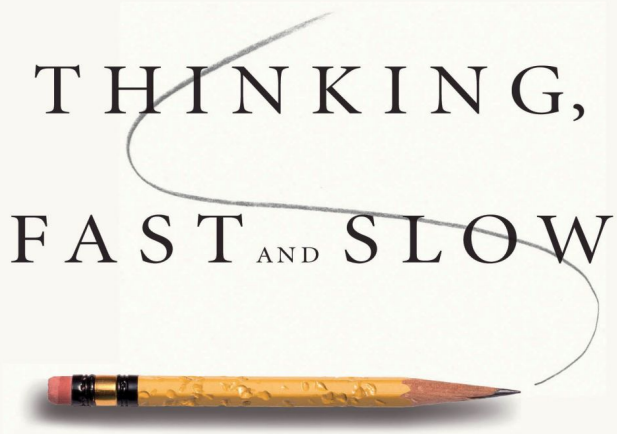
Where are we?


DONE
instruction following
code
math
...
factuality
safety
creativity
one or more stages
pretraining
w/ a language modeling objective
post-training
Supervised Fine-Tuning (SFT) / Reinforcement Learning (RL)

Kimi K3’s training pipeline



THINKING, FAST AND SLOW

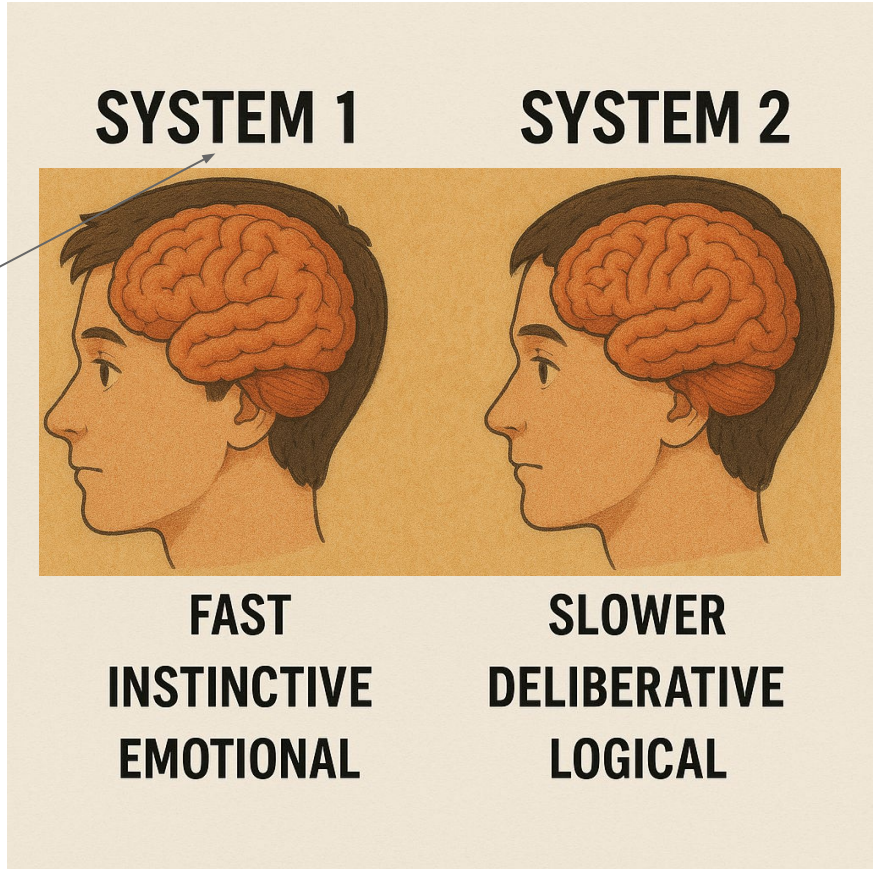


DANIEL KAHNEMAN

WINNER OF THE NOBEL PRIZE IN ECONOMICS

System 1 & System 2

TypeSafe's Jev



ChatGPT image

Chain-of-thought (CoT) prompting

CoT reasoning process

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ✗

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. ✓

OpenAI's o1

How it works

We trained these models to spend more time thinking through problems before they respond, much like a person would. Through training, they learn to refine their thinking process, try different strategies, and recognize their mistakes.

Similar to how a human may think for a long time before responding to a difficult question, o1 uses a chain of thought when attempting to solve a problem. Through reinforcement learning, o1 learns to hone its chain of thought and refine the strategies it uses. It learns to recognize and correct its mistakes. It learns to break down tricky steps into simpler ones. It learns to try a different approach when the current one isn't working. This process dramatically improves the model's ability to reason.

Can we infer its structure and what kinds of behaviors it rewards?

- The reward model likely provides scalar feedback that evaluates both **final answer** (correctness, usefulness, safety, clarity) and **reasoning process** (depth, coherence, reliability, safety awareness)
- Human annotators or automated evaluators review model outputs on reasoning tasks

What the reward model likely gives high scores for

1. Correct reasoning and factual accuracy

Outputs that reach correct conclusions through logically consistent reasoning receive higher rewards.

- On math or logic tasks: correct answers with verifiable reasoning.
- On open-ended questions: responses that are factually accurate and well supported.

2. Coherent, interpretable reasoning chains

Even if the chain of thought is not visible, internal reasoning steps that are internally consistent and lead to stable answers are likely rewarded.

- The model learns to reason step by step rather than jump to conclusions.
- RL training can penalize incoherent or contradictory internal reasoning trajectories.

3. Efficiency and confidence calibration

The model may get higher reward when it uses an appropriate amount of "thinking" (not too short or too long) and when it expresses uncertainty realistically.

- Correct but overly long reasoning could be slightly penalized.
- Overconfident incorrect answers could get low reward.

What the reward model likely gives high scores for (cont'd)

4. Safety, compliance, and harmlessness

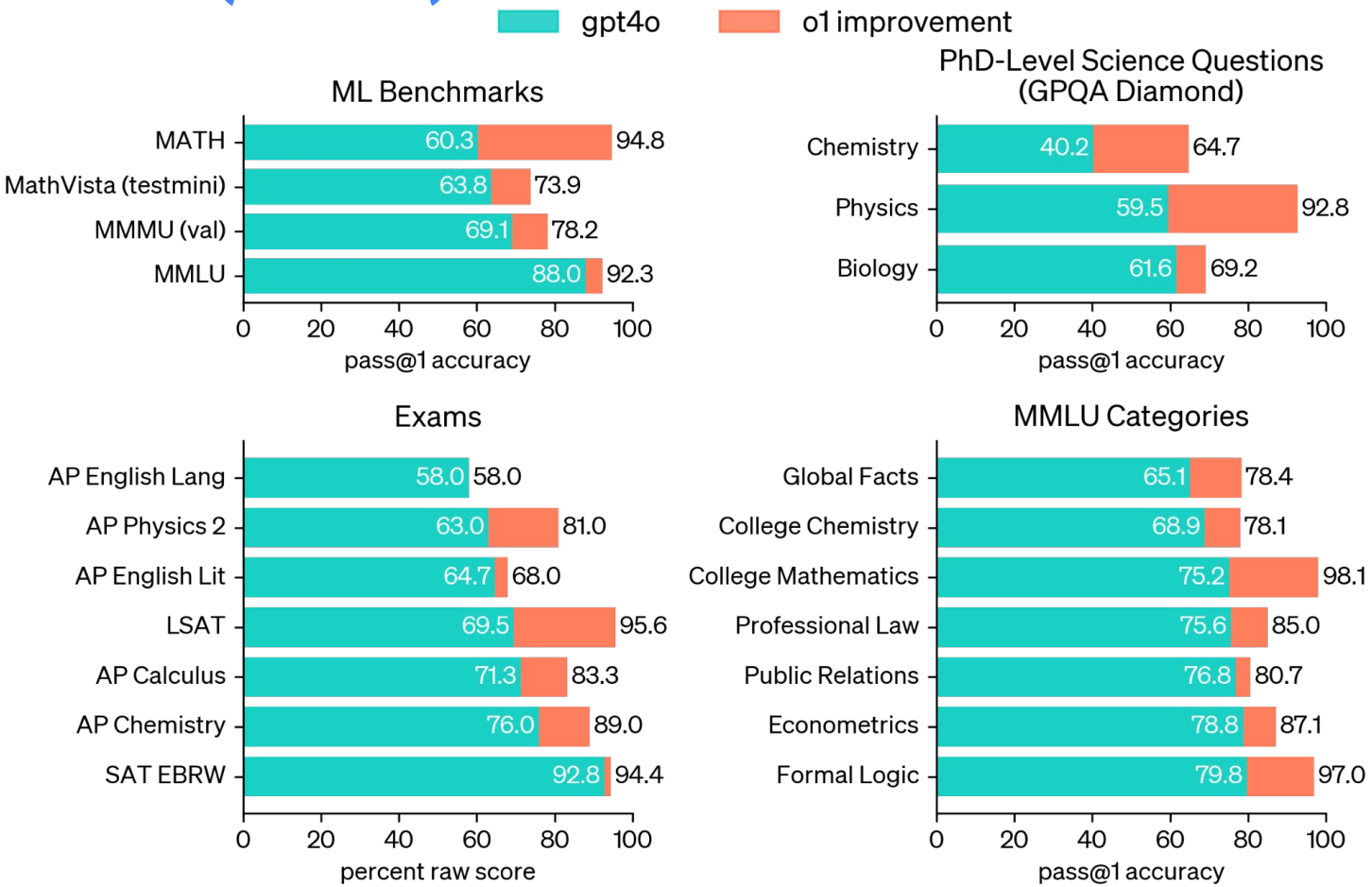
Outputs that adhere to safety policies and avoid unsafe or biased reasoning receive higher reward.

- The o1 system card emphasizes "deliberative alignment," meaning the model is rewarded for reasoning *about* safety before answering.
- Unsafe or policy-violating reasoning steps are penalized.

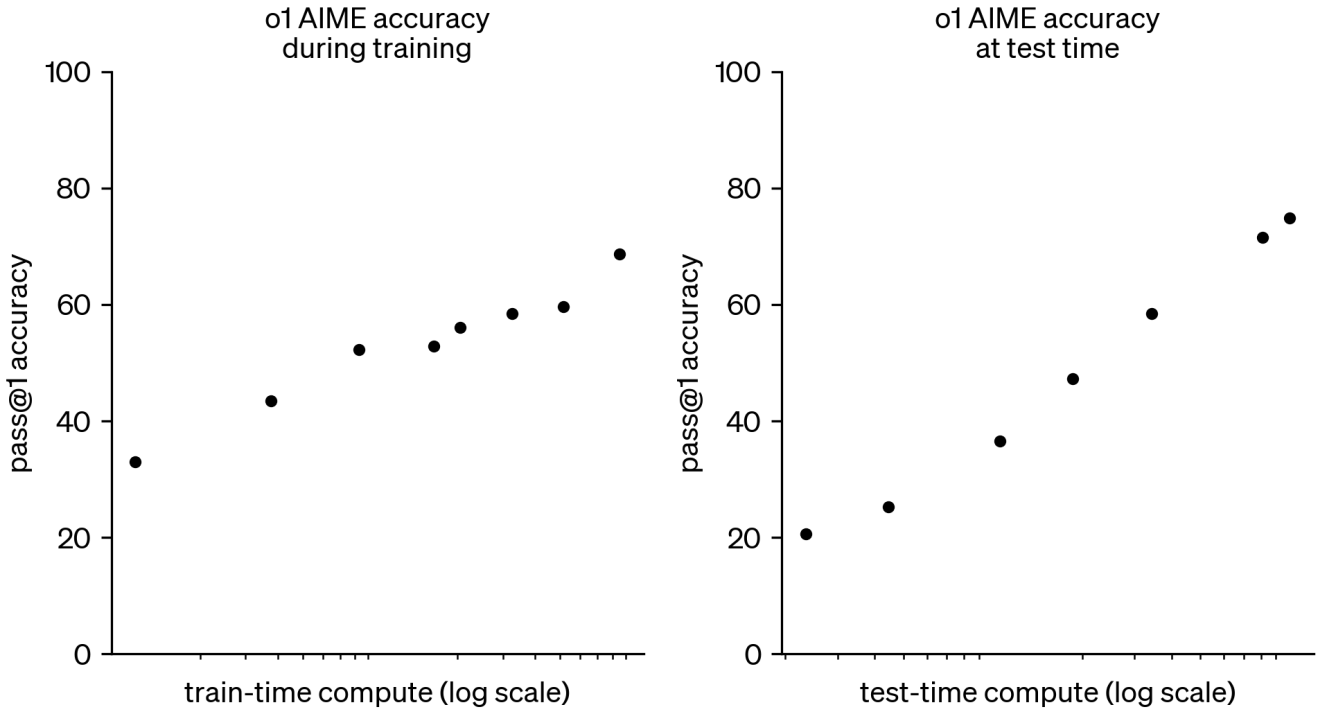
5. General helpfulness and clarity

The reward model also encourages clarity, helpful tone, and clear communication of reasoning results, similar to prior RLHF models.

OpenAI's o1 (cont'd)

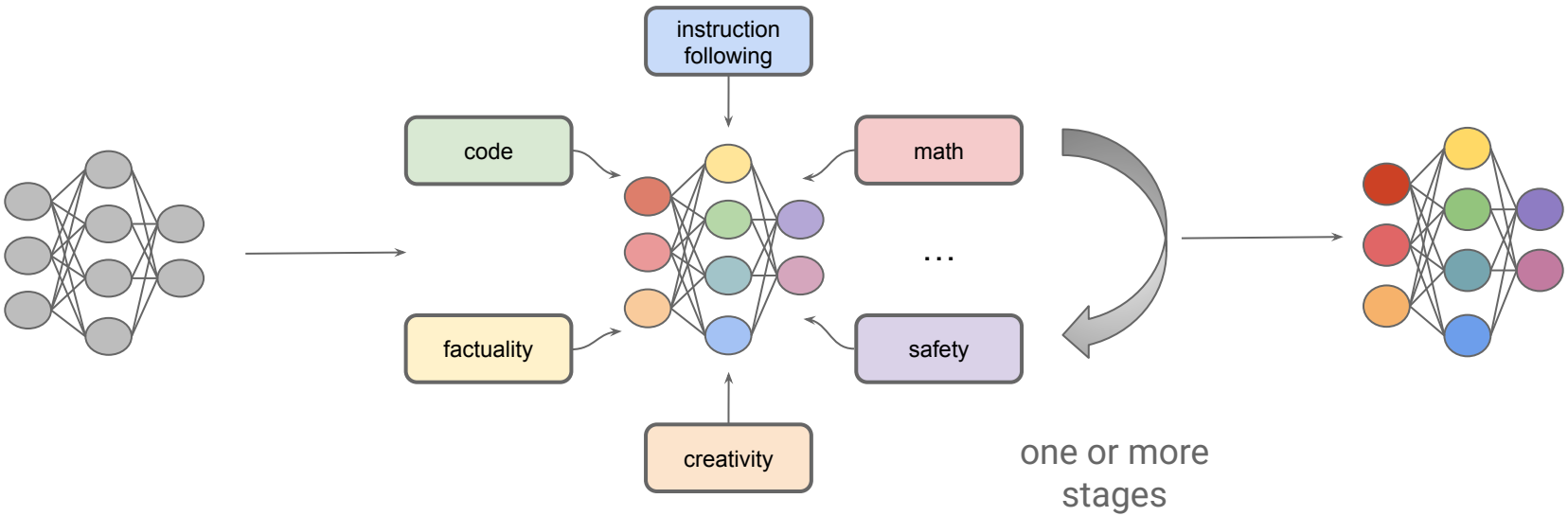


o1 performance smoothly improves with both train-time and test-time compute



https://openai.com/index/learning-to-reason-with-llms/

From System 1 to System 2



System 1

Supervised Fine-tuning and/or Reinforcement Learning on long Chain-of-Thought data

System 2

arXiv:2501.12948v1 [cs.CL] 22 Jan 2025

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI

research@deepseek.com

Abstract

We introduce our first-generation reasoning models, DeepSeek-R1-Zero and DeepSeek-R1. DeepSeek-R1-Zero, a model trained via large-scale reinforcement learning (RL) without supervised fine-tuning (SFT) as a preliminary step, demonstrates remarkable reasoning capabilities. Through RL, DeepSeek-R1-Zero naturally emerges with numerous powerful and intriguing reasoning behaviors. However, it encounters challenges such as poor readability, and language mixing. To address these issues and further enhance reasoning performance, we introduce DeepSeek-R1, which incorporates multi-stage training and cold-start data before RL. DeepSeek-R1 achieves performance comparable to OpenAI-o1-1217 on reasoning tasks. To support the research community, we open-source DeepSeek-R1-Zero, DeepSeek-R1, and six dense models (1.5B, 7B, 8B, 14B, 32B, 70B) distilled from DeepSeek-R1 based on Qwen and Llama.

DeepSeek-R1’s motivation

- develop reasoning capabilities **without any supervised data (labeled input-output pairs)**
- focus on their **self-evolution** through **a pure reinforcement learning process**

Reinforcement Learning Algorithm

- **Group Relative Policy Optimization** (GRPO)
  - Sample outputs from the current (old) policy and then optimize the policy model by maximizing the scalar reward given by a reward model

Reinforcement Learning from Verifiable Rewards (RLVR)

The reward is the source of the training signal, which decides the optimization direction of RL. To train DeepSeek-R1-Zero, we adopt a rule-based reward system that mainly consists of two types of rewards:

- **Accuracy rewards**: The accuracy reward model evaluates whether the response is correct. For example, in the case of math problems with deterministic results, the model is required to provide the final answer in a specified format (e.g., within a box), enabling reliable rule-based verification of correctness. Similarly, for LeetCode problems, a compiler can be used to generate feedback based on predefined test cases.
- **Format rewards**: In addition to the accuracy reward model, we employ a format reward model that enforces the model to put its thinking process between '<think>' and '</think>' tags.

Guided Chain-of-Thought (CoT) template

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within <think> </think> and <answer> </answer> tags, respectively, i.e., <think> reasoning process here </think> <answer> answer here </answer>. User: prompt. Assistant:

Group Relative Policy Optimization (GRPO)

Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right), \tag{1}$$

$$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \tag{2}$$

where ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - mean(\{r_1, r_2, \cdots, r_G\})}{std(\{r_1, r_2, \cdots, r_G\})}. \tag{3}$$

GRPO review

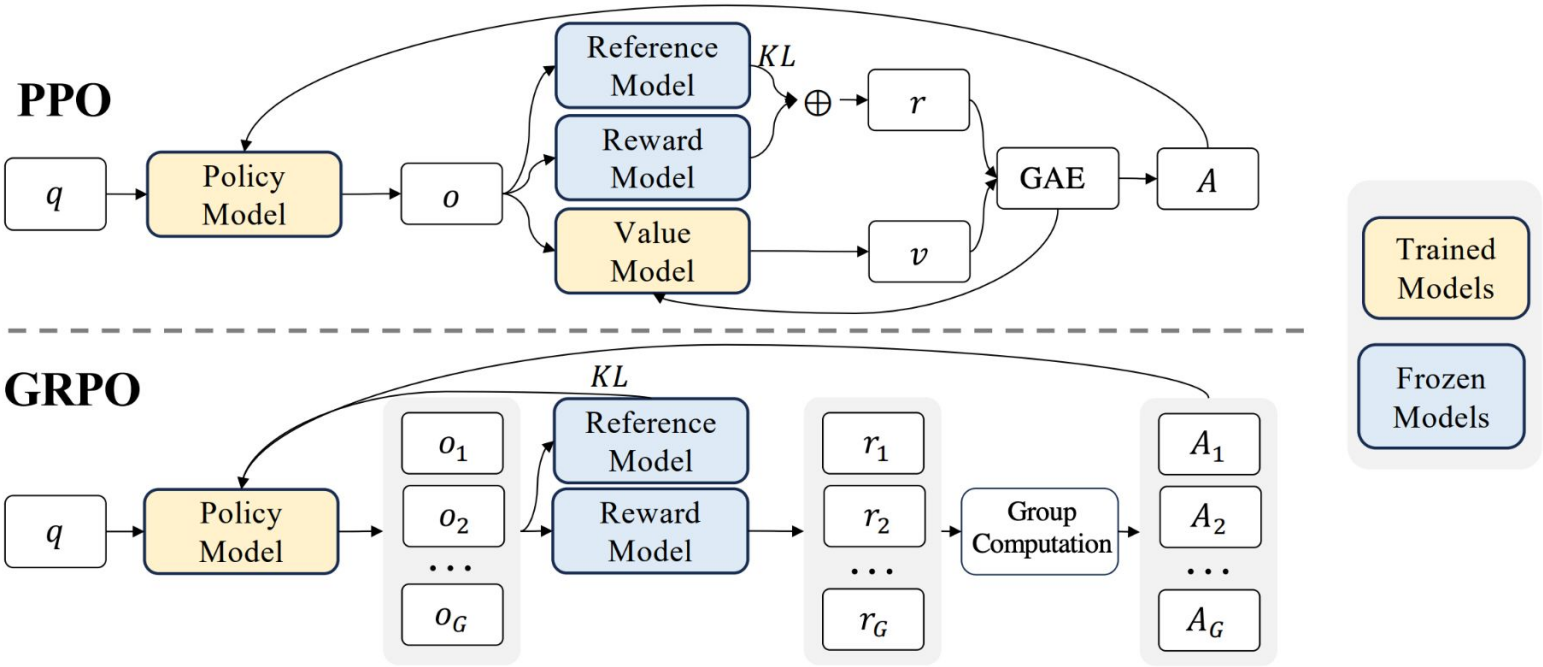


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

Group advantage estimation

The relative advantage for each output o_i is defined as:

$$A_i = \frac{r_i - \text{mean}(\{r_1, \ldots, r_G\})}{\text{std}(\{r_1, \ldots, r_G\})}.$$

This means that an output is considered “good” if its reward is higher than the group average.

This relative normalization stabilizes updates and avoids the need for an explicit baseline network.

Maximizing expected reward

The primary objective is to make outputs with higher rewards (from a reward model or human feedback) more likely under the updated policy.

This is achieved through the ratio term

$$\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)} A_i,$$

which increases the probability of samples with positive advantages $A_i > 0$ and decreases it for those with negative advantages $A_i < 0$.

Thus, GRPO pushes the model to favor responses that receive relatively higher scores within each group.

Maintaining policy stability

The **clipping function**

$$\text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}, 1-\epsilon, 1+\epsilon\right)$$

prevents large updates that could destabilize the model.

This is inherited from PPO: it ensures that the new policy does not deviate too aggressively from the previous one in a single training step.

Controlling divergence from the reference model

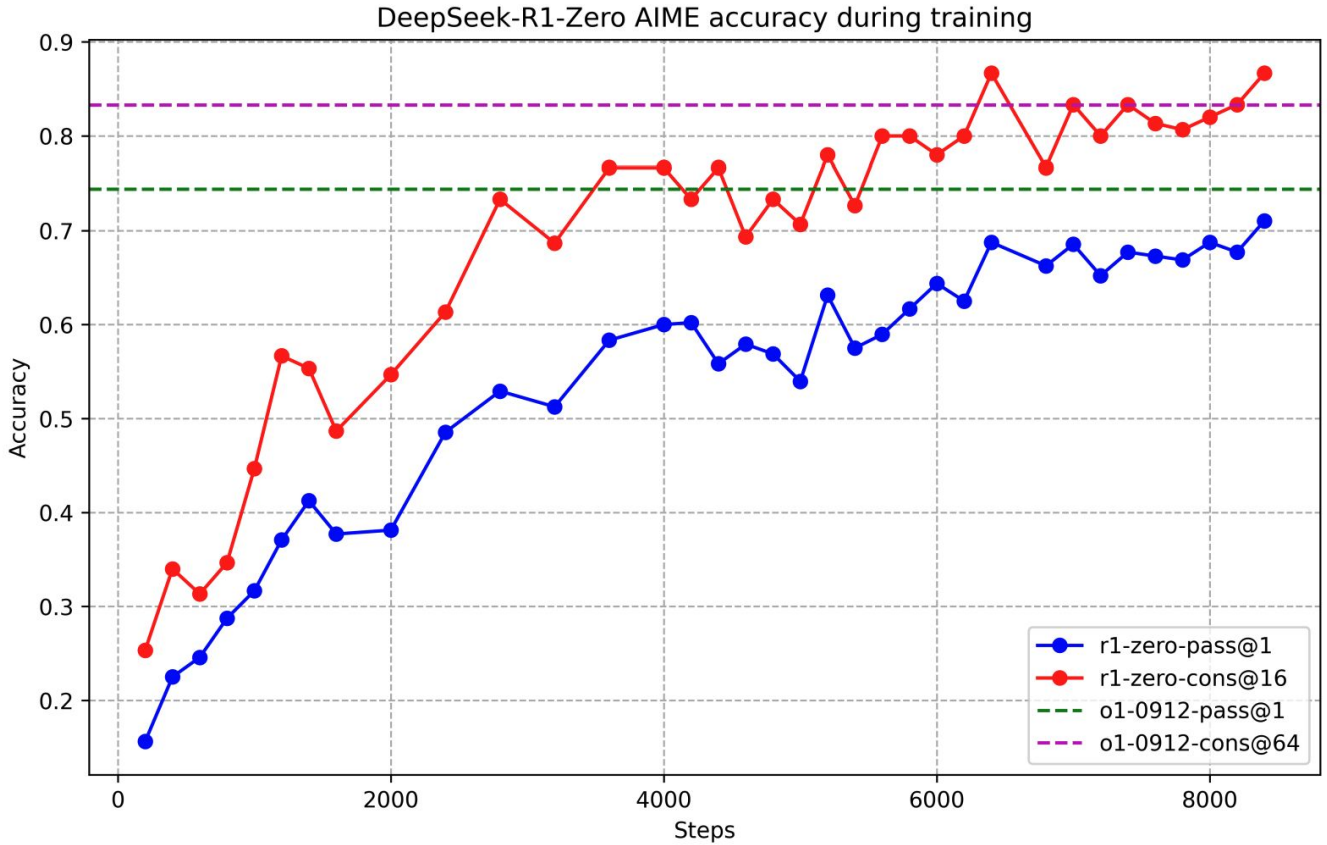
The **KL penalty term**

$$\beta D_{\mathrm{KL}}(\pi_\theta \| \pi_{\mathrm{ref}})$$

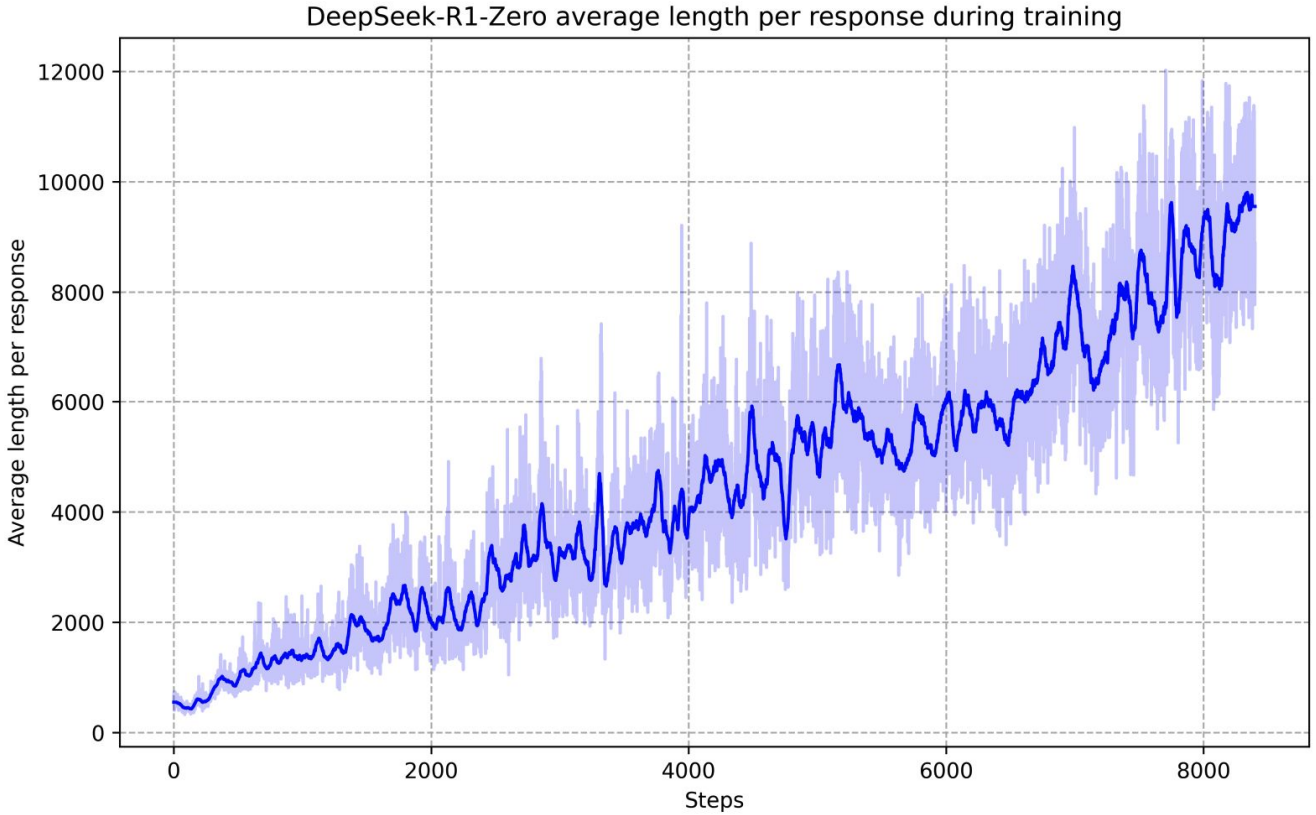
discourages the policy from diverging too far from a stable reference (often the supervised fine-tuned model).

This constraint keeps the learned behavior aligned with the base model's general language ability and prevents reward hacking or degeneration.

Accuracy of DeepSeek-R1-Zero during training



DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time



An interesting “aha moment” of DeepSeek-R1-Zero

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$

$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$

Rearrange to isolate the inner square root term:

$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$

$\cdots$

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$

We started with the equation:

$\sqrt{a - \sqrt{a + x}} = x$

First, let's square both sides:

$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$

Next, I could square both sides again, treating the equation: $\cdots$

$\cdots$

Problems with DeepSeek-R1-Zero

- It encounters challenges such as poor readability, and language mixing

Problems with DeepSeek-R1-Zero

- It encounters challenges such as poor readability, and language mixing

DeepSeek-R1

- Includes a small amount of cold-start data (thousands of long Chain-of-Thought (CoT) examples)

Post

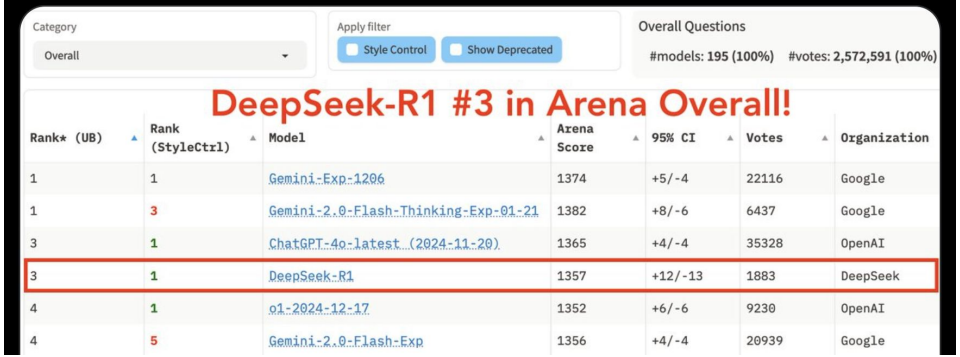
lmarena.ai (formerly lmsys.org)
@lmarena_ai

Breaking News: DeepSeek-R1 surges to the top-3 in Arena🐳!

Now ranked #3 Overall, matching the top reasoning model, o1, while being 20x cheaper and open-weight!

Highlights:
- #1 in technical domains: Hard Prompts, Coding, Math
- Joint #1 under Style Control
- MIT-licensed

A massive congrats to @deepseek_ai for this incredible milestone and gift to the community! More analysis below 👇



Rank* (UB)	Rank (StyleCtrl)	Model	Arena Score	95% CI	Votes	Organization
1	1	Gemini-Exp-1206	1374	+5/-4	22116	Google
1	3	Gemini-2.0-Flash-Thinking-Exp-01-21	1382	+8/-6	6437	Google
3	1	ChatGPT-4o-latest (2024-11-20)	1365	+4/-4	35328	OpenAI
3	1	DeepSeek-R1	1357	+12/-13	1883	DeepSeek
4	1	o1-2024-12-17	1352	+6/-6	9230	OpenAI
4	5	Gemini-2.0-Flash-Exp	1356	+4/-4	20939	Google

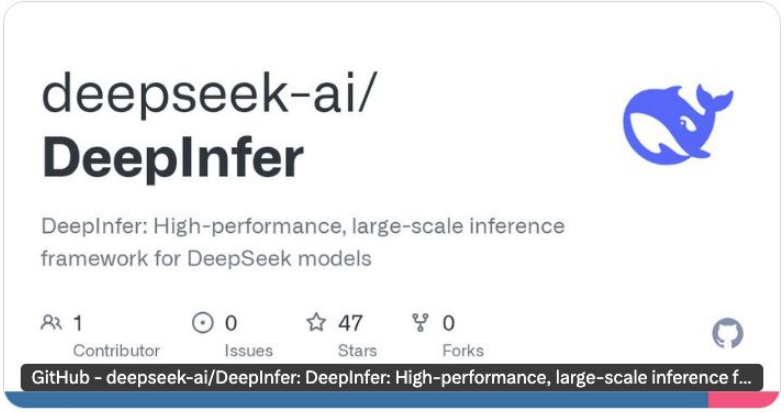
DeepSeek
@deepseek_ai

🚀 Introducing DeepInfer, our in-house inference framework, now fully open source!

Powering DeepSeek-V4.1-Flash at scale with high throughput and low latency. We're sharing our production stack to help the community lower compute costs, reduce energy use, and make AI more affordable.

🔗 github.com/deepseek-ai/De...

1/4

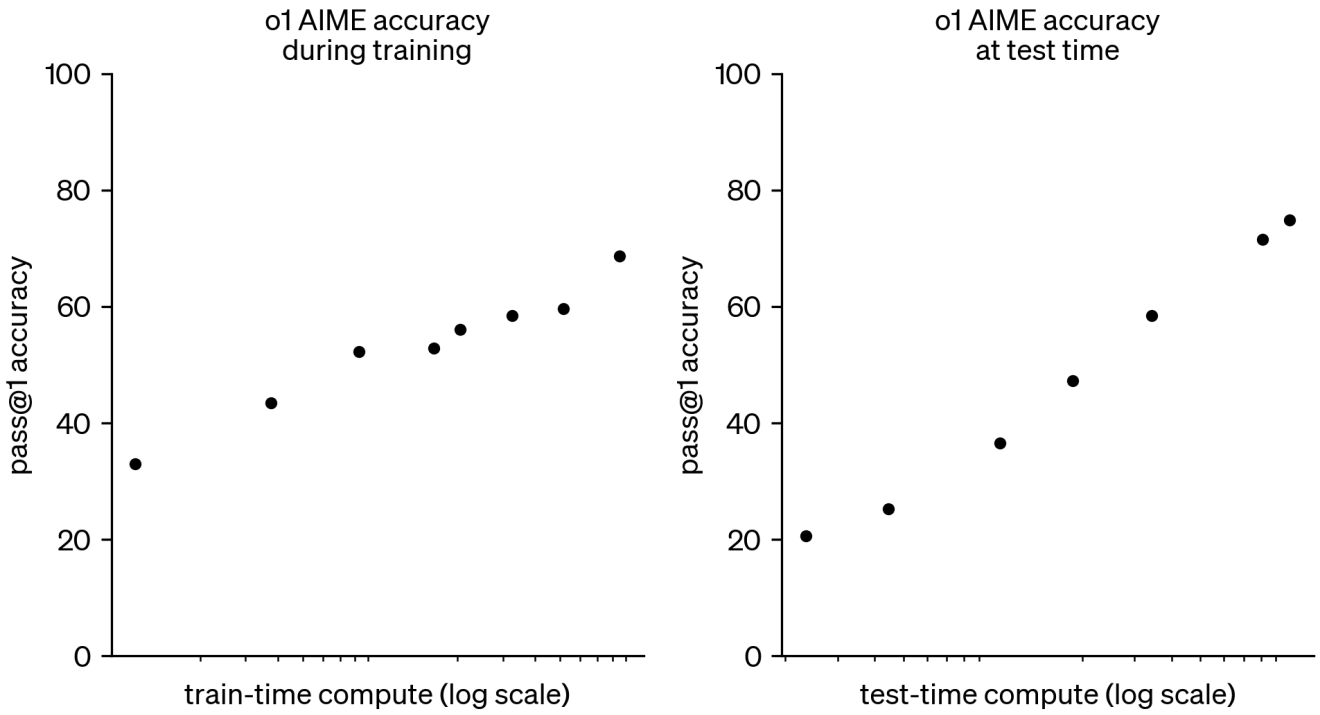


From github.com

5:30 AM · Oct 20, 2026 · **2.1M** Views

1K 4.1K 23K 8.4K

Test-time scaling



https://openai.com/index/learning-to-reason-with-llms/

Test-time scaling methods

- **Parallel (repeated sampling)**
  - multiple solution attempts (run independently)
  - chooses the most frequent or the best response
  - Brown et al. (2024); Irvine et al. (2023); Levi (2024)
- **Sequential**
  - later computations depend on earlier ones (e.g., a long reasoning trace)
  - allows it to refine each attempt based on previous outcomes
  - Muennighoff et al.(2025); Snell et al. (2024); Hou et al. (2025); Lee et al. (2025)

Large Language Monkeys: Scaling Inference Compute with Repeated Sampling

Bradley Brown*†‡, Jordan Juravsky*†, Ryan Ehrlich*†, Ronald Clark‡, Quoc V. Le§, Christopher Ré†, and Azalia Mirhoseini†§

†Department of Computer Science, Stanford University
‡University of Oxford
§Google DeepMind

bradley.brown@cs.ox.ac.uk, jbj@stanford.edu, ryanehrlich@cs.stanford.edu, ronald.clark@cs.ox.ac.uk, qvl@google.com, chrismre@stanford.edu, azalia@stanford.edu

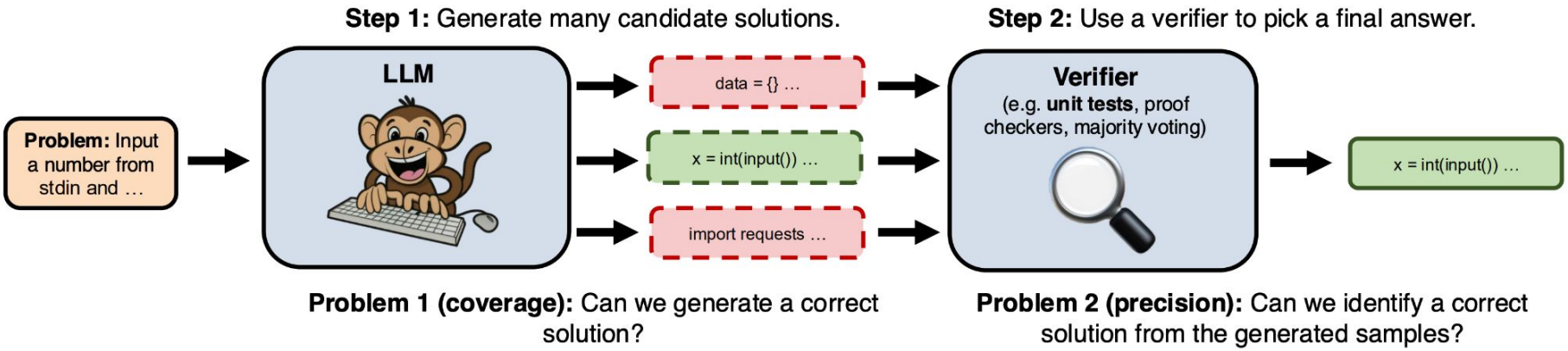


Figure 1: The repeated sampling procedure that we follow in this paper. 1) We generate many independent candidate solutions for a given problem by sampling from an LLM with a positive temperature. 2) We use a domain-specific verifier (ex. unit tests for code) to select a final answer from the generated samples.

Measuring coverage

$$\text{pass@}k = \frac{1}{\#\text{ of problems}} \sum_{i=1}^{\#\text{ of problems}} \left[1 - \frac{\binom{N-C_i}{k}}{\binom{N}{k}} \right]$$

$$\text{pass@}k = \frac{1}{\#\text{ of problems}} \sum_{i=1}^{\#\text{ of problems}} \left[1 - \frac{\binom{N-C_i}{k}}{\binom{N}{k}}\right]$$

- N: Total number of completions (outputs) generated per problem.
- C_i: Number of **correct** completions among the N for problem i.
- $\binom{N}{k}$: Number of ways to choose k outputs from N.
- $\binom{N-C_i}{k}$: Number of ways to choose k outputs *all of which are incorrect* (i.e., chosen only from the $N - C_i$ incorrect ones).
- So, $\frac{\binom{N-C_i}{k}}{\binom{N}{k}}$ is the **probability** that you choose **only incorrect outputs** when sampling k outputs from the N total.
- Subtracting this from 1 gives the probability that **at least one** correct solution is in the top-k.

Coverage increases as we scale the number of samples

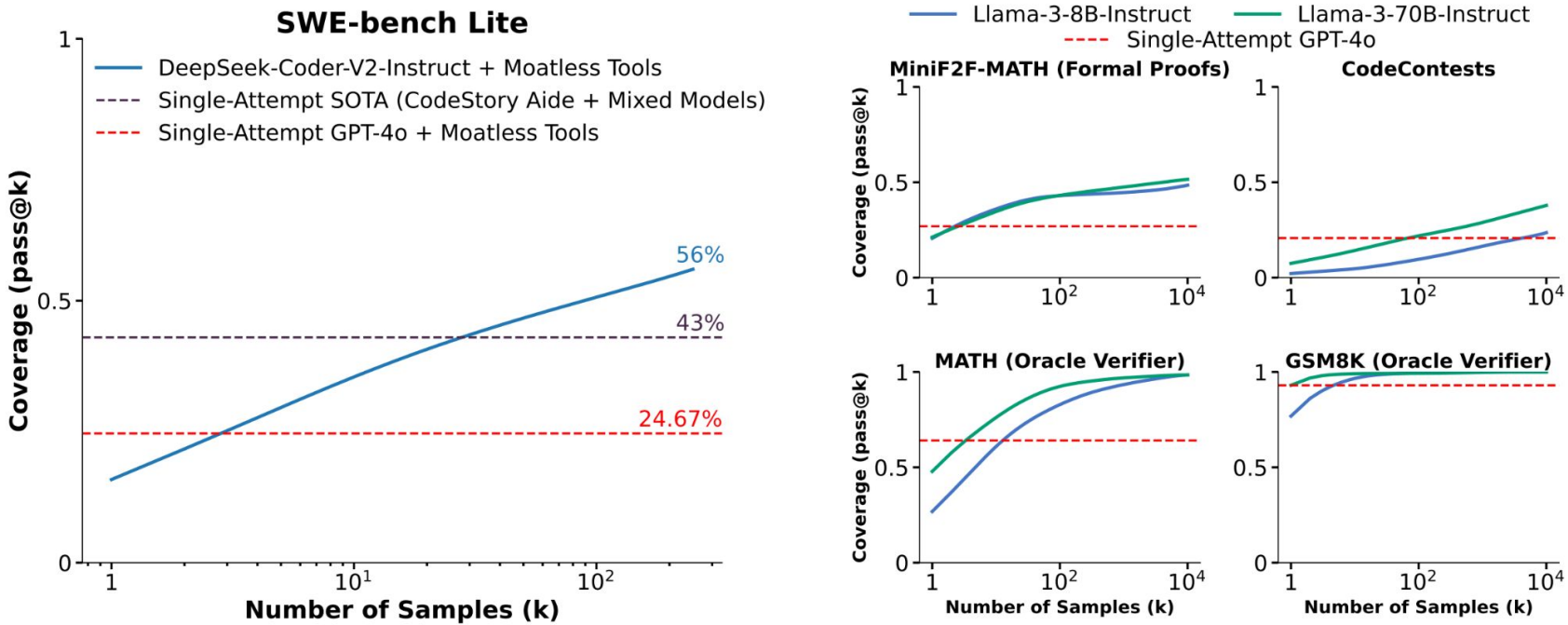


Figure 2: Across five tasks, we find that coverage (the fraction of problems solved by at least one generated sample) increases as we scale the number of samples. Notably, using repeated sampling, we are able to increase the solve rate of an open-source method from 15.9% to 56% on SWE-bench Lite.

Scaling inference time compute via repeated sampling leads to consistent coverage gains

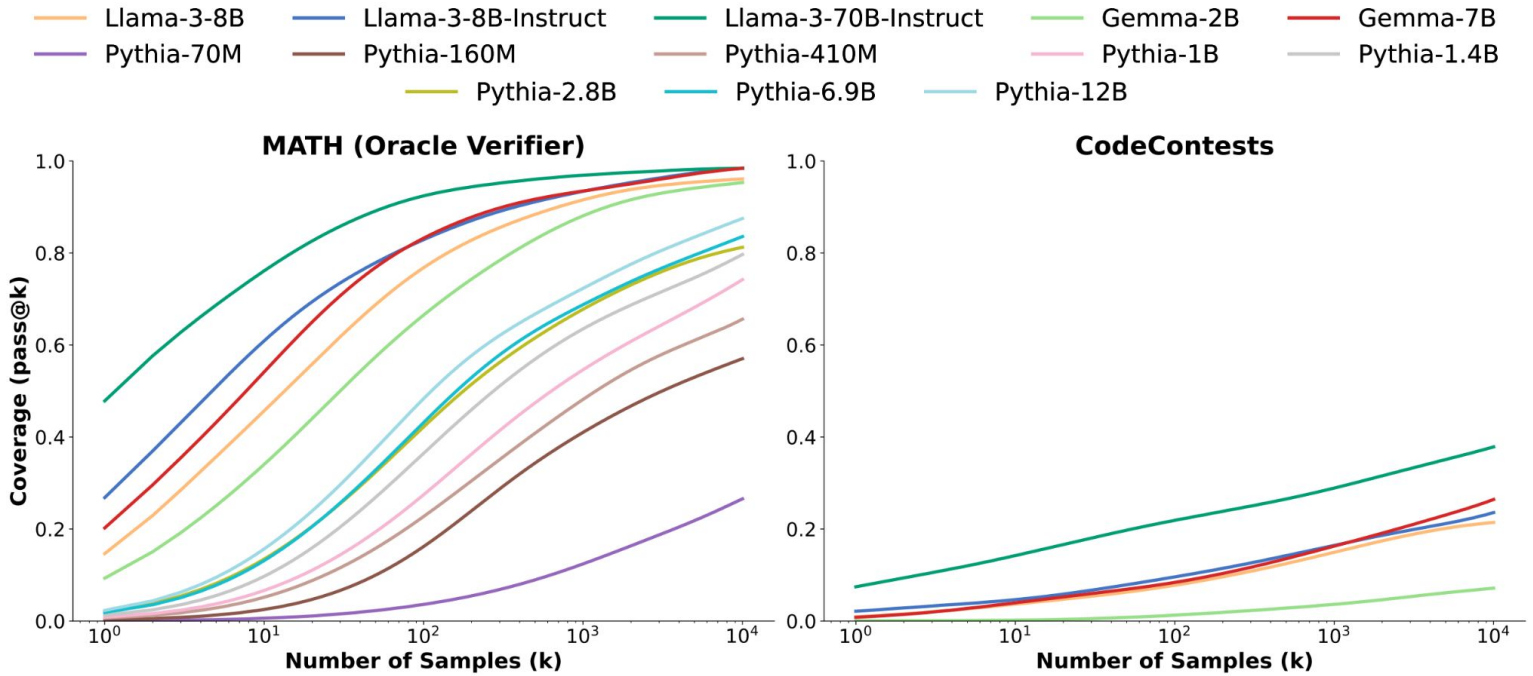


Figure 3: Scaling inference time compute via repeated sampling leads to consistent coverage gains across a variety of model sizes (70M-70B), families (Llama, Gemma and Pythia) and levels of post-training (Base and Instruct models).

Inference FLOPs

$$\text{FLOPsPerToken}(\text{ContextLen}) \approx 2 * (\text{NumParameters} + 2 * \text{NumLayers} * \text{TokenDim} * \text{ContextLen})$$

$$\text{TotalInferenceFLOPs} \approx \left(\sum_{t=1}^{\text{NumPromptTokens}} \text{FLOPsPerToken}(t) \right) + \left(\sum_{t=1}^{\text{NumDecodeTokens}} \text{FLOPsPerToken}(t + \text{NumPromptTokens}) * \text{NumCompletions} \right)$$

Ideal model size depends on the task, compute budget, and coverage requirements

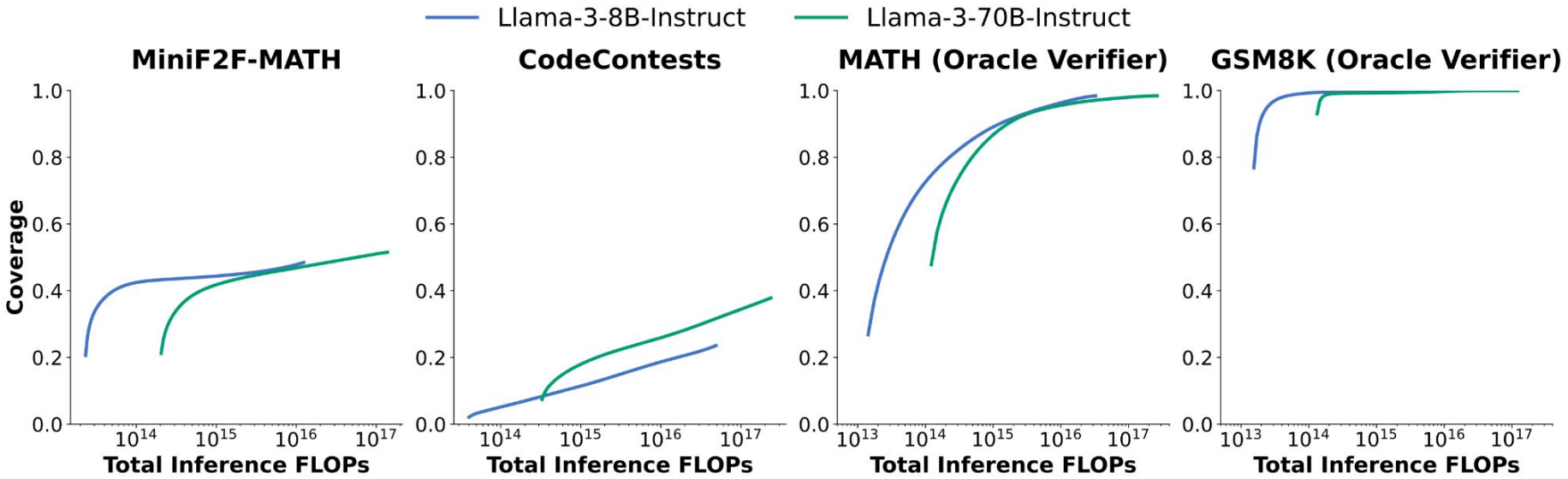


Figure 4: Comparing cost, measured in number of inference FLOPs, and coverage for Llama-3-8B-Instruct and Llama-3-70B-Instruct. We see that the ideal model size depends on the task, compute budget, and coverage requirements. Note that Llama-3-70B-Instruct does not achieve 100% coverage on GSM8K due to an incorrectly labelled ground truth answer: see Appendix E.

API cost

Model	Cost per attempt (USD)	Number of attempts	Issues solved (%)	Total cost (USD)	Relative total cost
DeepSeek-Coder-V2-Instruct	0.0072	5	29.62	10.8	1x
GPT-4o	0.13	1	24.00	39	3.6x
Claude 3.5 Sonnet	0.17	1	26.70	51	4.7x

Table 1: Comparing API cost (in US dollars) and performance for various models on the SWE-bench Lite dataset using the Moatless Tools agent framework. When sampled more, the open-source DeepSeek-Coder-V2-Instruct model can achieve the same issue solve-rate as closed-source frontier models for under a third of the price.

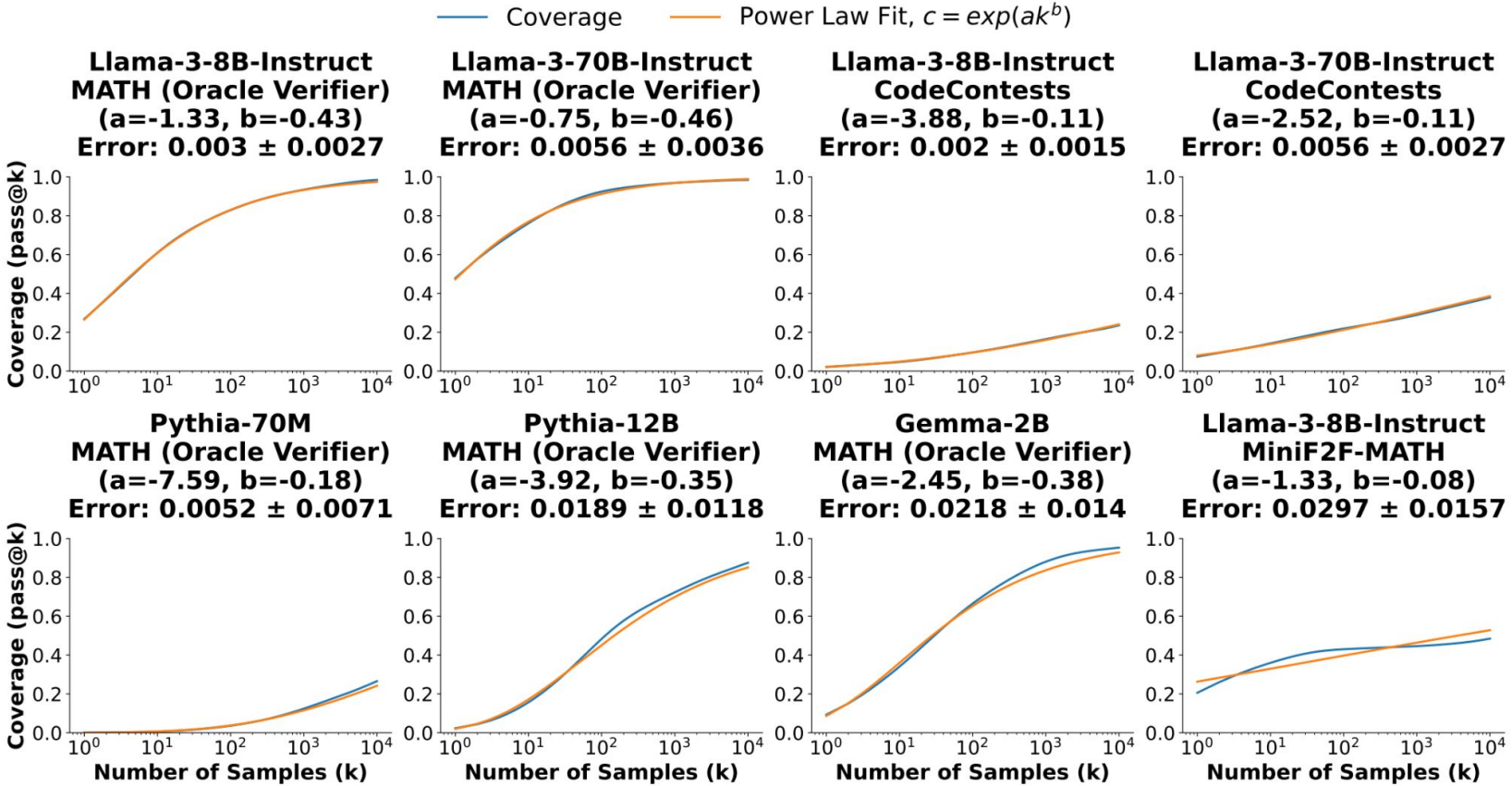


Figure 5: The relationship between coverage and the number of samples can be modelled with an exponentiated power law for most tasks and models. We highlight that some curves, such as Llama-3-8B-Instruct on MiniF2F-MATH, do not follow this trend closely. We show the mean and standard deviation of the error between the coverage curve and the power law fit across 100 evenly sampled points on the log scale.

We model the logarithm of coverage c as a function of the number of samples k:

$$\log(c) \approx ak^b$$

where $a, b \in \mathbb{R}$ are parameters to be fitted.

To directly predict coverage c, we exponentiate both sides:

$$c \approx \exp(ak^b)$$

The relationship between coverage and the number of samples modelled with an exponentiated power law

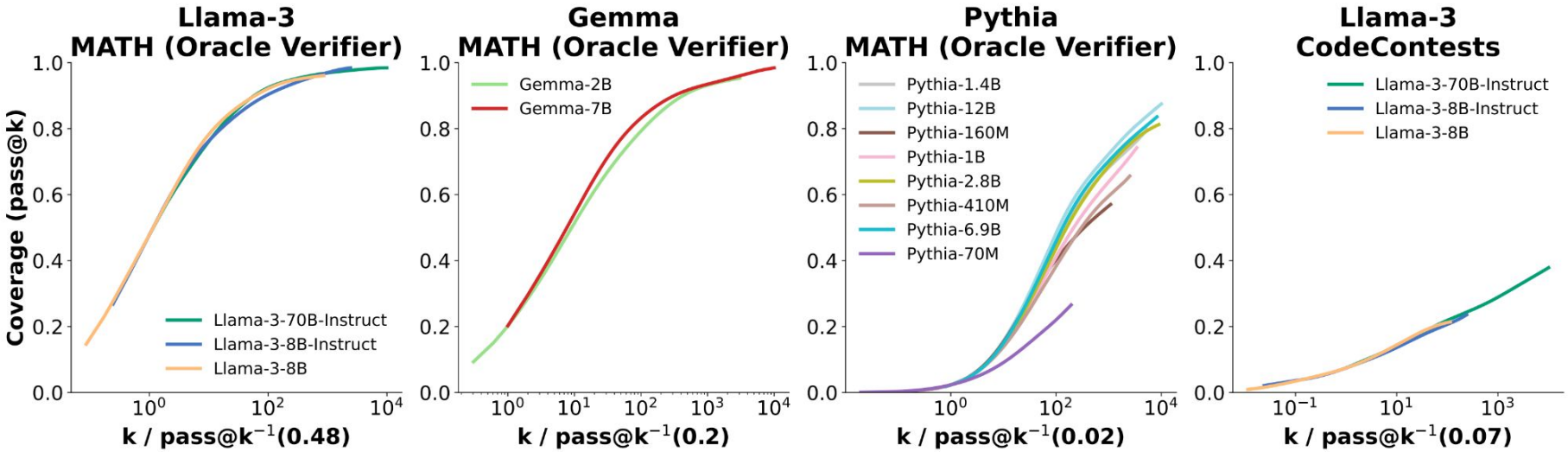


Figure 6: Overlaying the coverage curves from different models belonging to the same family. We perform this overlay by horizontally shifting every curve (with a logarithmic x-axis) so that all curves pass through the point $(1, c)$. We pick c to be the maximum pass@1 score over all models in the plot. We note that the similarity of the curves post-shifting shows that, within a model family, sampling scaling curves follow a similar shape.

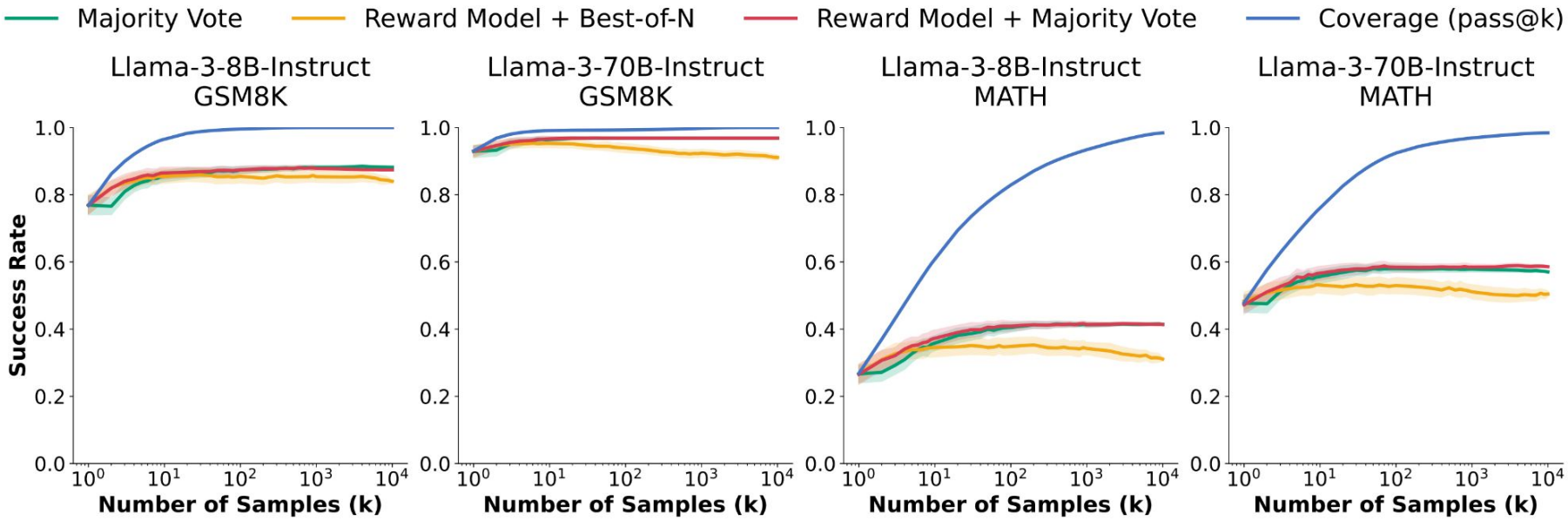


Figure 7: Comparing coverage (performance with an oracle verifier) to mainstream methods available for picking the correct answer (majority voting, reward model selection and reward model majority voting) as we increase the number of samples. Although near-perfect coverage is achieved, all sample selection methods fail to reach the coverage upper bound and saturate before reaching 100 samples. For every k value, we calculate the metric on 100 subsets of size k then plot the mean and one standard deviation across subsets.

Pass@1	# Problems	# CoT Graded	Correct CoT	Incorrect CoT	Incorrect Ground Truth
0-10%	5	15	11	1	1 problem, 3 CoTs
10-25%	10	30	27	3	0 problems
25-75%	29	30	28	2	0 problems
75-100%	84	30	30	0	0 problems

Table 2: Human evaluation of the validity of the Chain-of-Thought reasoning in Llama-3-8B-Instruct answers to GSM8K problems. 3 chains of thought were graded per problem. Even for difficult questions, where the model only gets $\leq 10\%$ of samples correct, the CoTs almost always follow valid logical steps. For the model generations and human labels, see here.

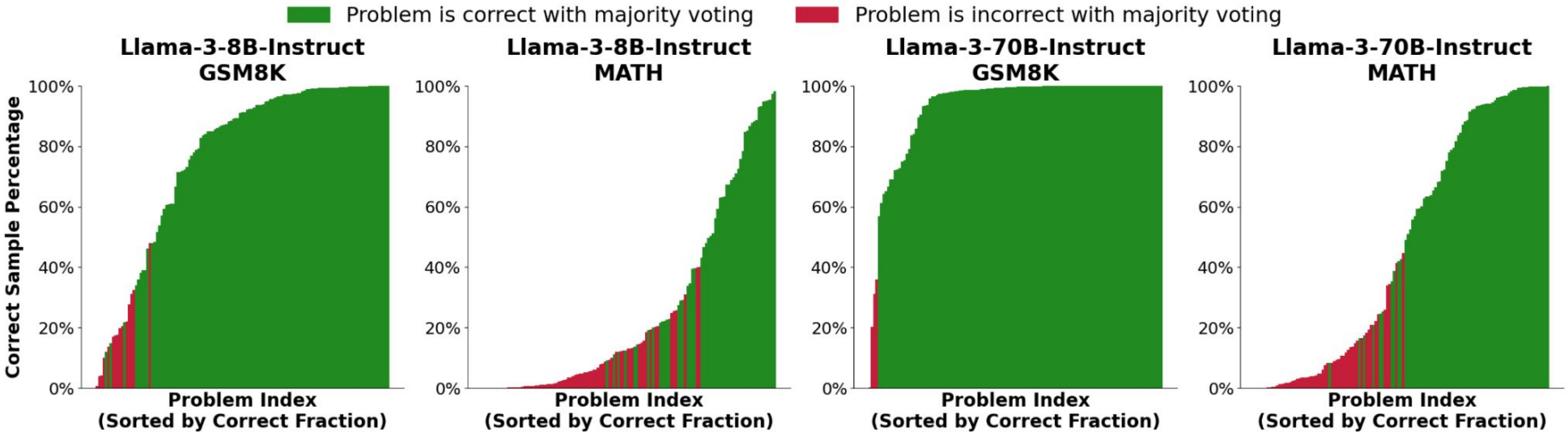


Figure 8: Bar charts showing the fraction of samples (out of 10,000 samples) that are correct, for each problem in the subsets of GSM8K and MATH we evaluate on. There is one bar per problem, and the height of the bar corresponds to the fraction of samples that arrive at the correct answer. Bars are green if self-consistency picked the correct answer and are red otherwise. We highlight that there are many problems with correct solutions, where the correct solutions are sampled infrequently.

s1: Simple test-time scaling

Niklas Muennighoff [*1 3 4] **Zitong Yang** [*1] **Weijia Shi** [*2 3] **Xiang Lisa Li** [*1] **Li Fei-Fei** [1] **Hannaneh Hajishirzi** [2 3]
Luke Zettlemoyer [2] **Percy Liang** [1] **Emmanuel Candès** [1] **Tatsunori Hashimoto** [1]

Test-time scaling with s1-32B

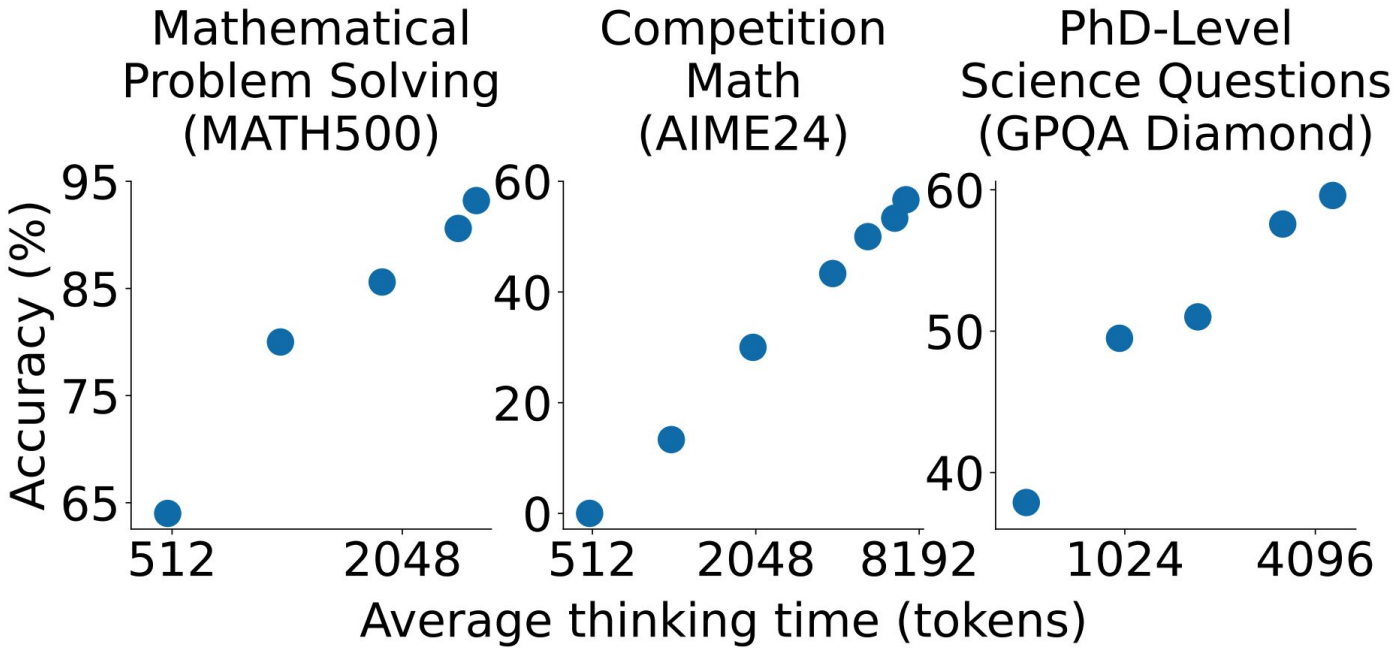
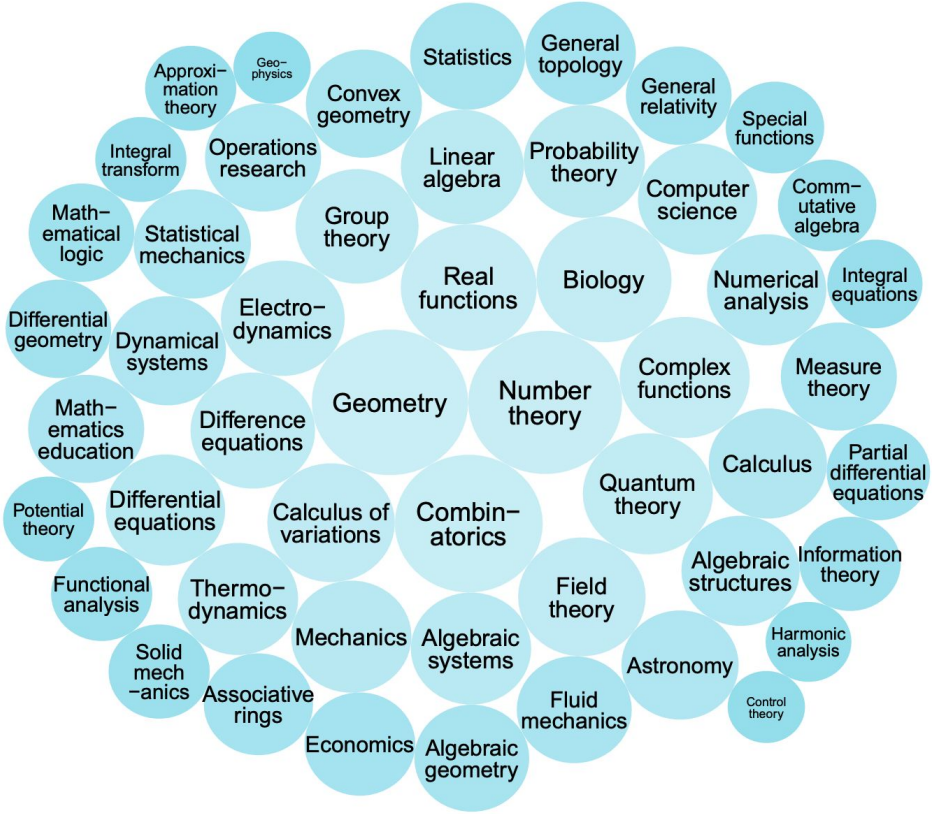
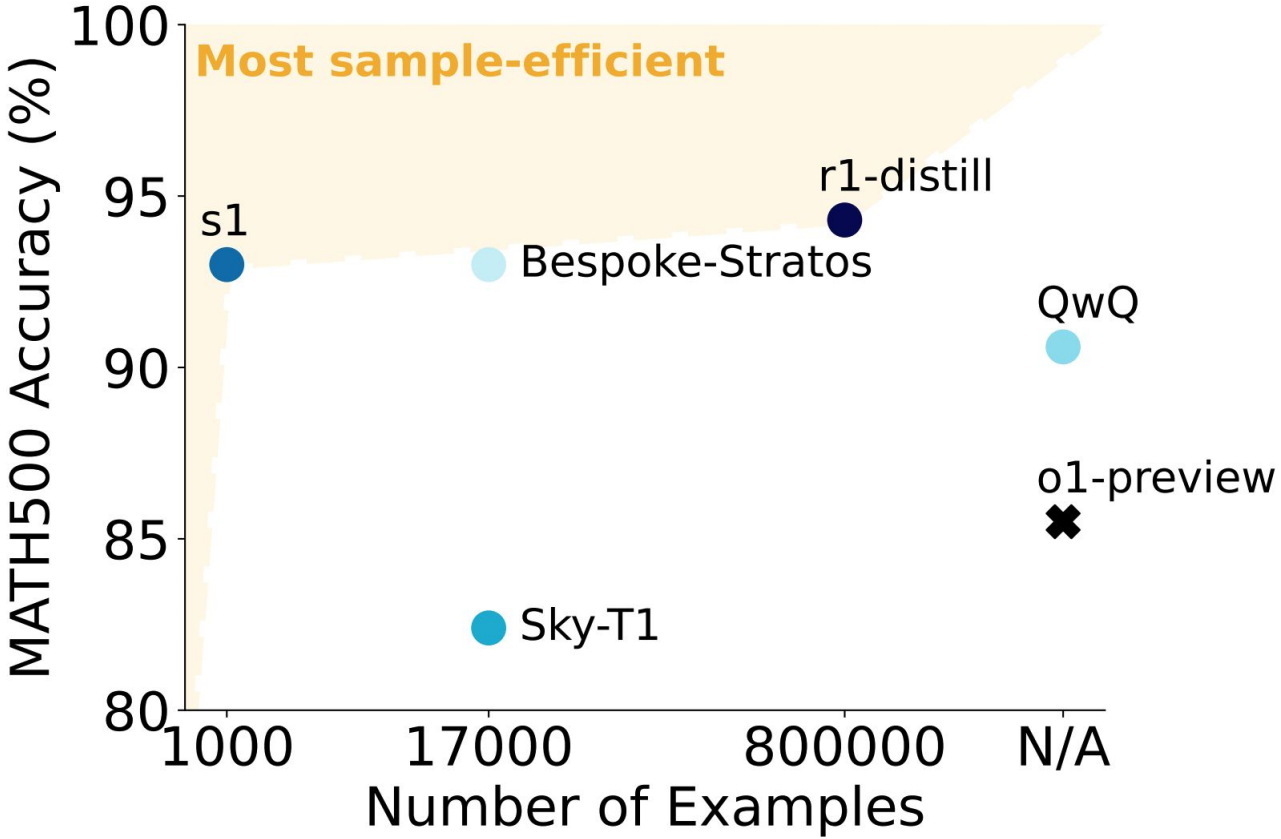


Figure 1. **Test-time scaling with s1-32B.** We benchmark **s1-32B** on reasoning-intensive tasks and vary test-time compute.

s1K is a dataset of 1,000 high-quality, diverse, and difficult questions with reasoning traces.



s1-32B is on the sample-efficiency frontier (Pareto)



Budget forcing

How many r in raspberry? **Question**

Let's break down the process of counting the letter 'r' in the word "raspberry" ...
* First letter: 'r' - This is an 'r', count = 1.
* Second letter: 'a' - Not an 'r', count remains 1 ...
* Sixth letter: 'e' - Not an 'r', count remains 1.
* Seventh letter: 'r' - This is an 'r', count = 2.
* Eighth letter: 'y' - Not an 'r', count remains 2 ...
The number of 'r's in "raspberry" is 2.
Wait, let's re-read the question carefully. It asks "How many r in raspberry?" ... * r - a - s - p - b - e - r - r - y ... * First 'r' ... * Second 'r' ... * Third 'r' ... Count = 3 ... **Reasoning trace**

My initial answer of 2 was incorrect due to a quick reading of the word. **Final Answer:** The final answer is $\boxed{3}$ **Response**

For each method under consideration, we evaluate a set of tasks $a \in \mathcal{A}$ by varying the test-time compute on a fixed benchmark (e.g., AIME24). This yields a piecewise linear function f, where the x-axis represents compute (measured in thinking tokens) and the y-axis corresponds to accuracy.

Control

$$\text{Control} = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \mathbb{I}(a_{\min} \leq a \leq a_{\max})$$

This measures the proportion of evaluations whose test-time compute (in thinking tokens) falls within a specified range $[a_{\min}, a_{\max}]$. In practice, we often constrain only $a_{\max}$. As thinking tokens reflect test-time compute, this metric quantifies how well a method supports controllability over computational cost. We report Control as a percentage, with 100% indicating perfect control.

Scaling

$$\text{Scaling} = \frac{1}{\binom{|\mathcal{A}|}{2}} \sum_{\substack{a,b \in \mathcal{A} \\ b > a}} \frac{f(b) - f(a)}{b - a}$$

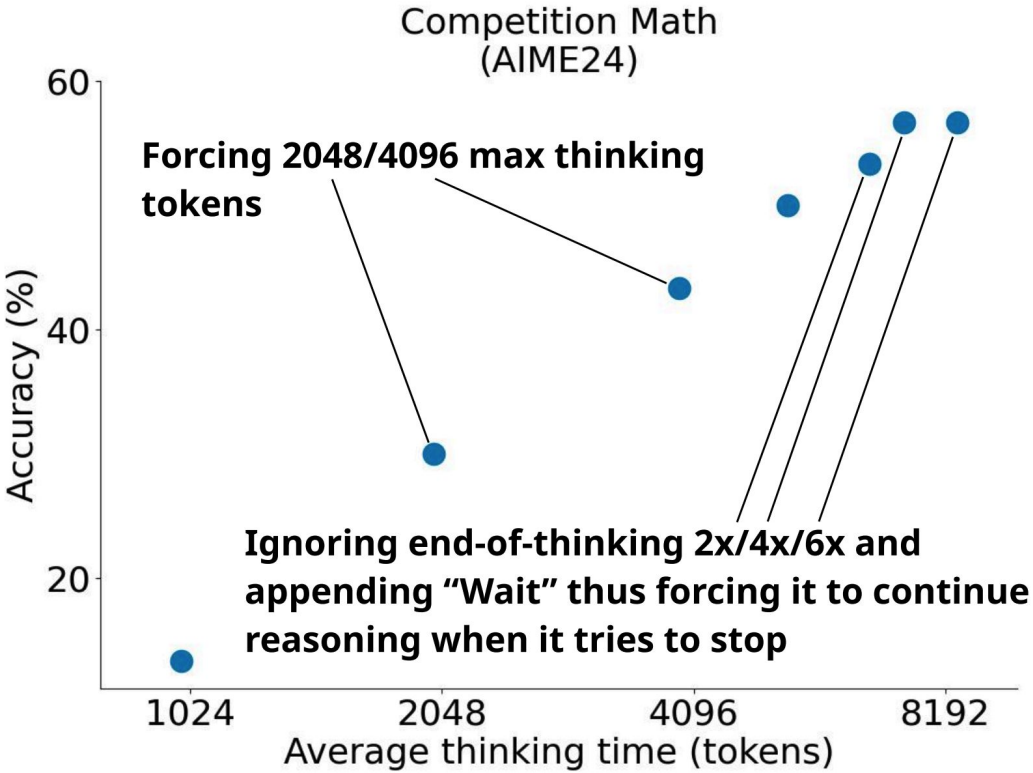
This captures the average slope of the piecewise linear function f, representing the relationship between compute and accuracy. A positive slope is necessary, and higher values indicate better scaling behavior.

Performance

$$\text{Performance} = \max_{a \in \mathcal{A}} f(a)$$

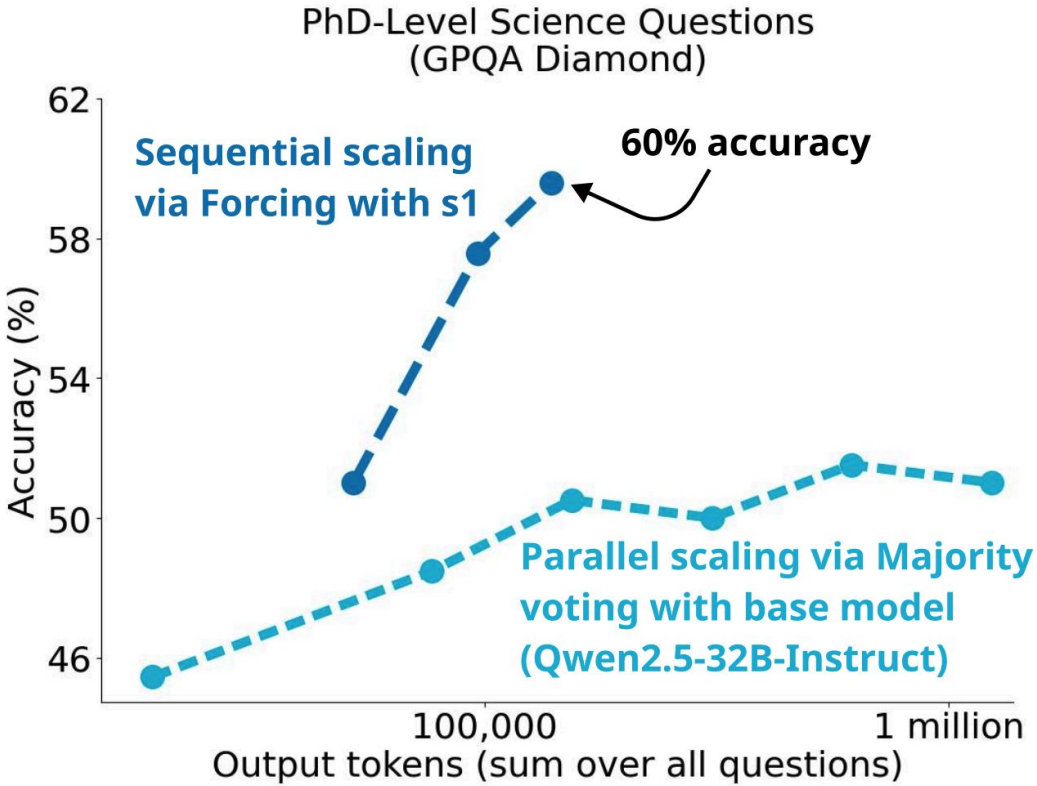
This is the highest accuracy achieved across all levels of test-time compute. In the limit, a method with consistently positive scaling will approach 100% performance.

Budget forcing shows clear scaling trends and extrapolates to some extent



(a) Sequential scaling via budget forcing

Parallel scaling via majority voting



(b) Parallel scaling via majority voting

s1-32B is a strong open reasoning model

Model	# ex.	AIME 2024	MATH 500	GPQA Diamond
API only				
o1-preview	N.A.	44.6	85.5	73.3
o1-mini	N.A.	70.0	90.0	60.0
o1	N.A.	**74.4**	**94.8**	**77.3**
Gemini 2.0 Flash Think.	N.A.	60.0	N.A.	N.A.
Open Weights				
Qwen2.5-32B-Instruct	N.A.	26.7	84.0	49.0
QwQ-32B	N.A.	50.0	90.6	54.5
r1	≫800K	**79.8**	**97.3**	**71.5**
r1-distill	800K	72.6	94.3	62.1
Open Weights and Open Data				
Sky-T1	17K	43.3	82.4	56.8
Bespoke-32B	17K	**63.3**	93.0	58.1
s1 w/o BF	**1K**	50.0	92.6	56.6
s1-32B	**1K**	56.7	**93.0**	**59.6**

s1K data ablations

Model	AIME 2024	MATH 500	GPQA Diamond
1K-random	36.7 [-26.7%, -3.3%]	90.6 [-4.8%, 0.0%]	52.0 [-12.6%, 2.5%]
1K-diverse	26.7 [-40.0%, -10.0%]	91.2 [-4.0%, 0.2%]	54.6 [-10.1%, 5.1%]
1K-longest	33.3 [-36.7%, 0.0%]	90.4 [-5.0%, -0.2%]	59.6 [-5.1%, 10.1%]
59K-full	53.3 [-13.3%, 20.0%]	92.8 [-2.6%, 2.2%]	58.1 [-6.6%, 8.6%]
s1K	50.0	93.0	57.6

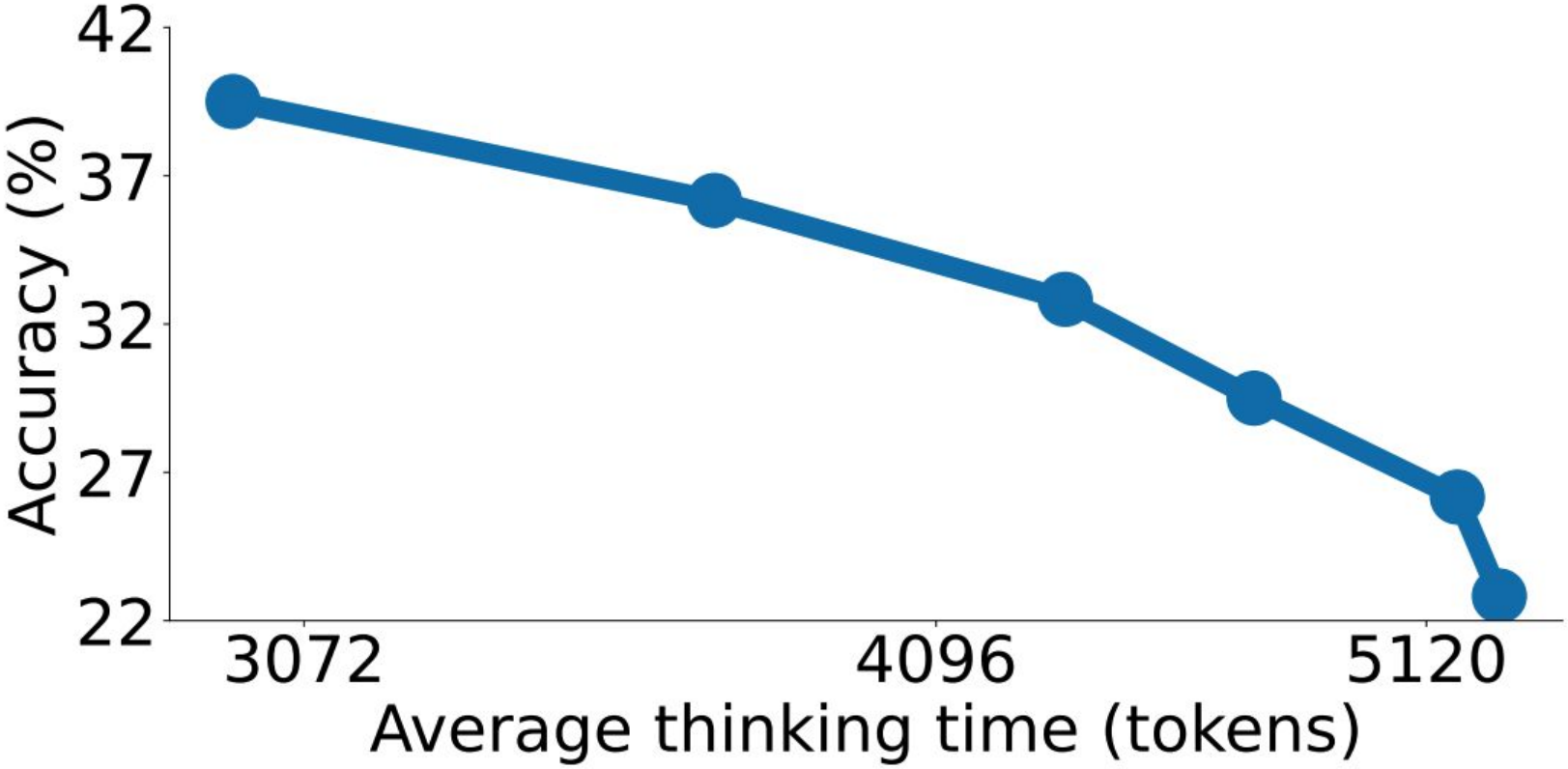
Ablations on methods to scale test-time compute

Method	Control	Scaling	Performance	$\lvert\mathcal{A}\rvert$
BF	**100%**	15	**56.7**	5
TCC	40%	-24	40.0	5
TCC + BF	**100%**	13	40.0	5
SCC	60%	3	36.7	5
SCC + BF	**100%**	6	36.7	5
CCC	50%	**25**	36.7	2
RS	**100%**	-35	40.0	5

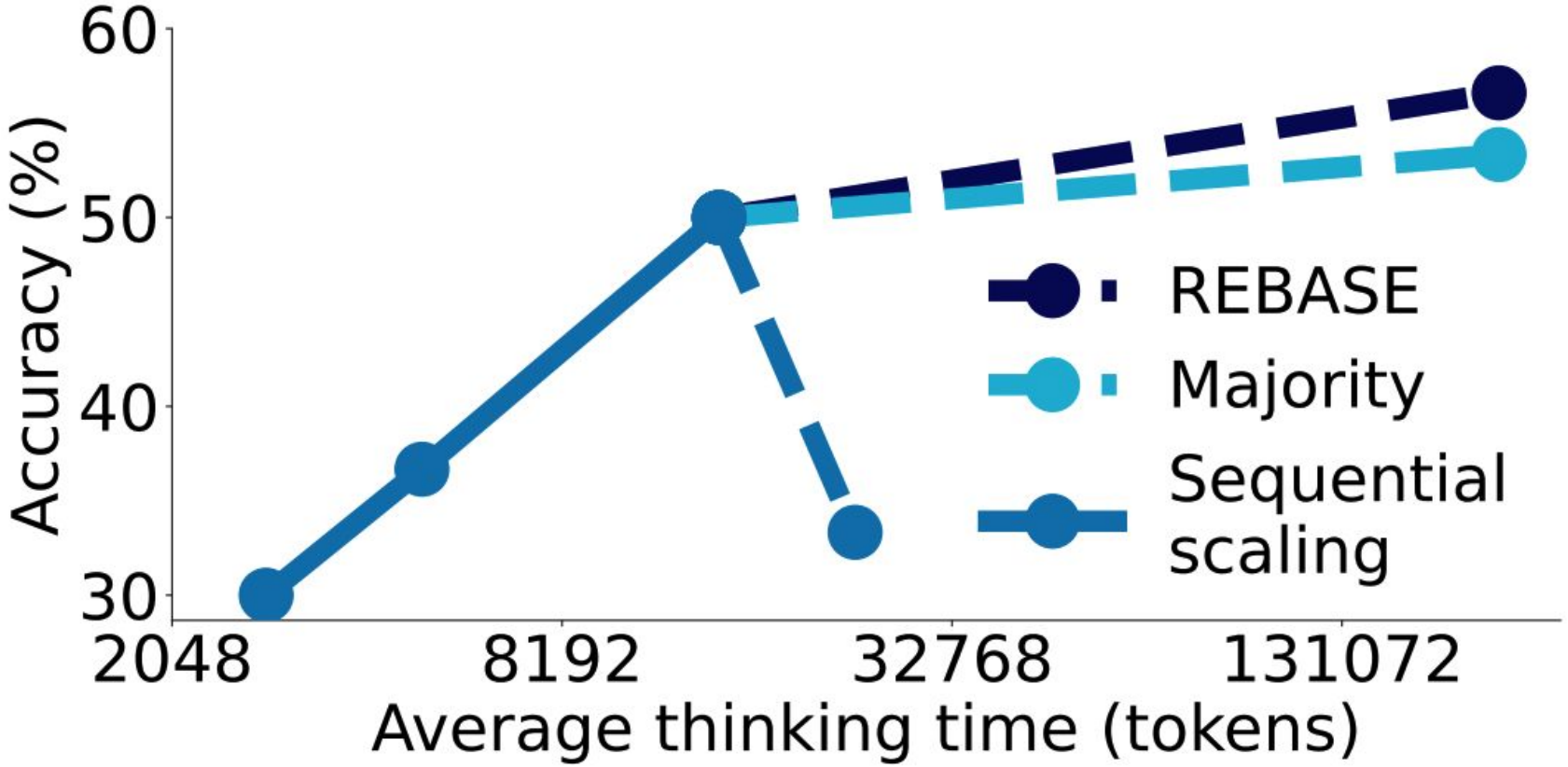
Budget forcing extrapolation ablations

Model	AIME 2024	MATH 500	GPQA Diamond
No extrapolation	50.0	**93.0**	57.6
2x without string	50.0	90.2	55.1
2x “Alternatively”	50.0	92.2	**59.6**
2x “Hmm”	50.0	**93.0**	**59.6**
2x “Wait”	**53.3**	**93.0**	**59.6**

Rejection sampling



Augmenting s1 with REBASE (process reward model)



Why does supervised fine-tuning on just 1,000 samples lead to such performance gains?

- We hypothesize that the model is already exposed to large amounts of reasoning data during pretraining which spans trillions of tokens.
- Thus, the ability to perform reasoning is already present in our model.
- Our sample-efficient fine-tuning stage just activates it and we scale it further at test time with budget forcing.

Superficial Alignment Hypothesis

- LIMA: Less is more for alignment (Zhou et al., 2023)
  - 1,000 examples can be sufficient to align a model to adhere to user preferences

LIMO: Less is More for Reasoning

Yixin Ye* **Zhen Huang*** **Yang Xiao** **Ethan Chern** **Shijie Xia** **Pengfei Liu†**

SJTU, SII, GAIR

Superficial Alignment Hypothesis

- LIMA: Less is more for alignment (Zhou et al., 2023)
  - 1,000 examples can be sufficient

- LIMO: even competition-level complex reasoning abilities can be effectively elicited through minimal but curated training samples

- LIMO: a promising technical pathway toward AGI - any sophisticated reasoning capability, no matter how complex, could potentially be activated with minimal samples given two key conditions:

  - (1) sufficient domain knowledge embedded during pre-training
  - (2) optimal cognitive reasoning chains for activation

Categorizing the reasoning chains into five

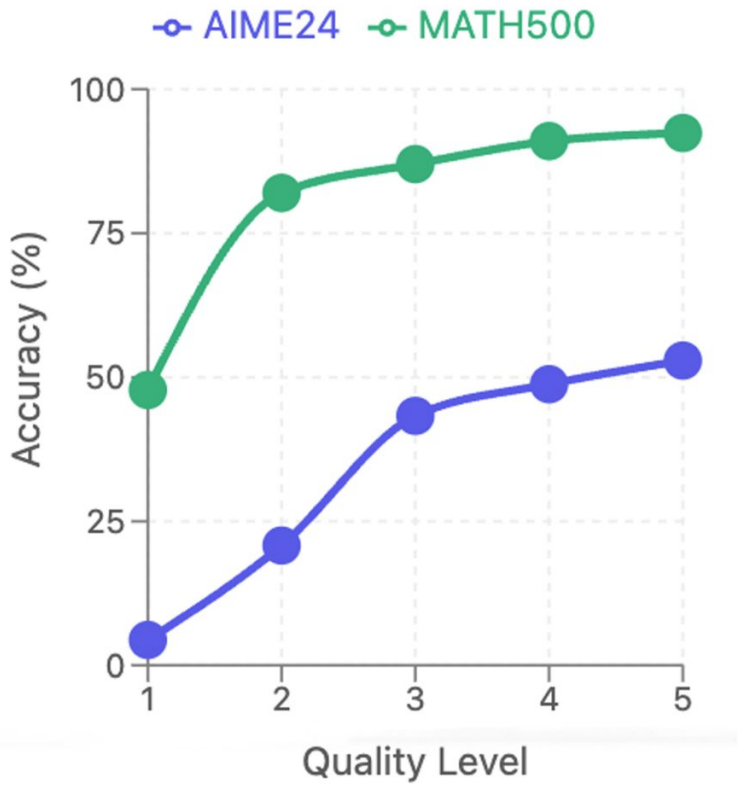
How well the reasoning steps were organized, whether important logical transitions were properly explained, and if the solution included self-verification steps

- **L5:** excellent organization with clear, well-explained steps and thorough self-verification
- **L4:** well-structured but perhaps with slightly less rigorous checking
- **L3:** decent organization but sometimes skipped over explaining crucial logical leaps
- **L2:** often provided abbreviated reasoning without much explanation
- **L1:** just listed basic steps with minimal elaboration and rarely included any verification

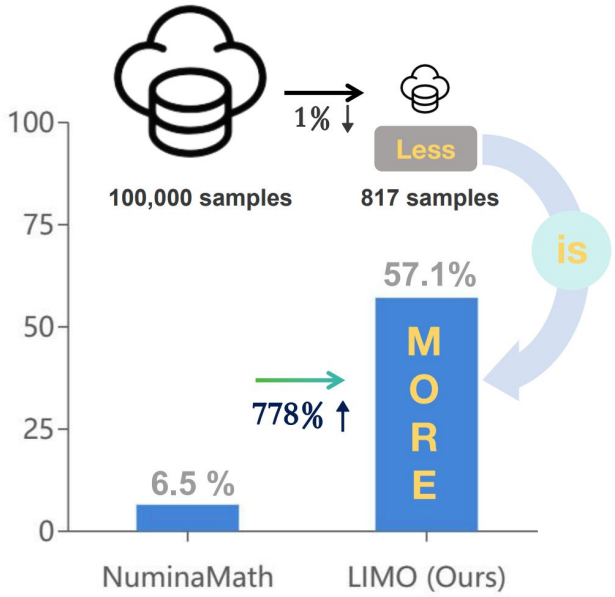
Statistical analysis of different quality levels

Data Quality Level	Avg. Tokens per response	Avg. Lines per response	Top 10 Frequently Occurring Keywords (in order)
Level 1	230	9.21	since, however, number, let, thus, which, get, two, triangle, theta
Level 2	444.88	50.68	number, need, times, which, find, list, thus, since, triangle, sum
Level 3	4956.11	375.60	**perhaps**, **alternatively**, **consider**, number, **wait**, which, sides, need, equal, seems
Level 4	4726.97	354.87	**wait**, which, number, **perhaps**, therefore, let, since, **maybe**, sides, two
Level 5	5290.26	239.29	**wait**, therefore, which, number, since, lets, two, sides, let, **maybe**

Comparison of models trained on reasoning chains of different quality levels



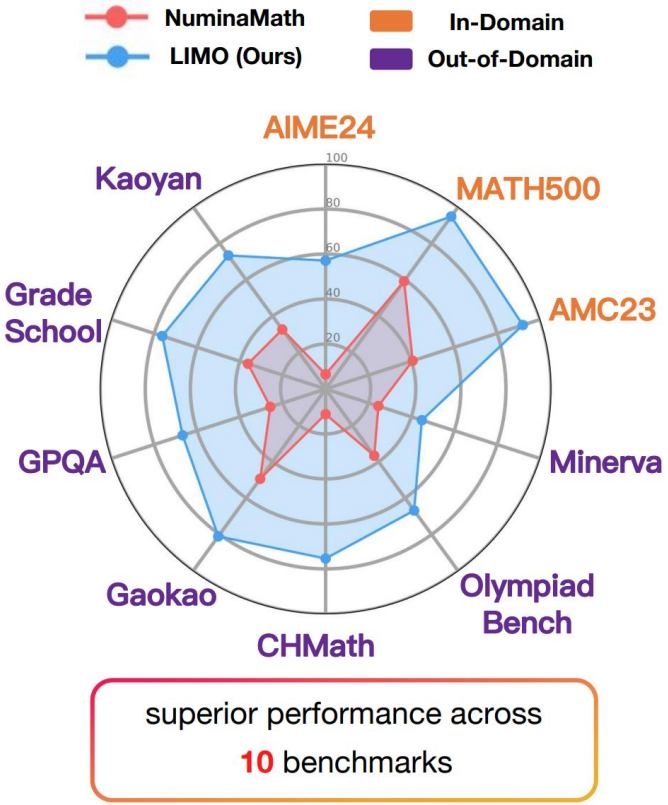
LIMO achieves substantial improvement over NuminaMath with fewer samples



completely **same** backbone

1% data → **778%** gain on AIME24 (pass@1)

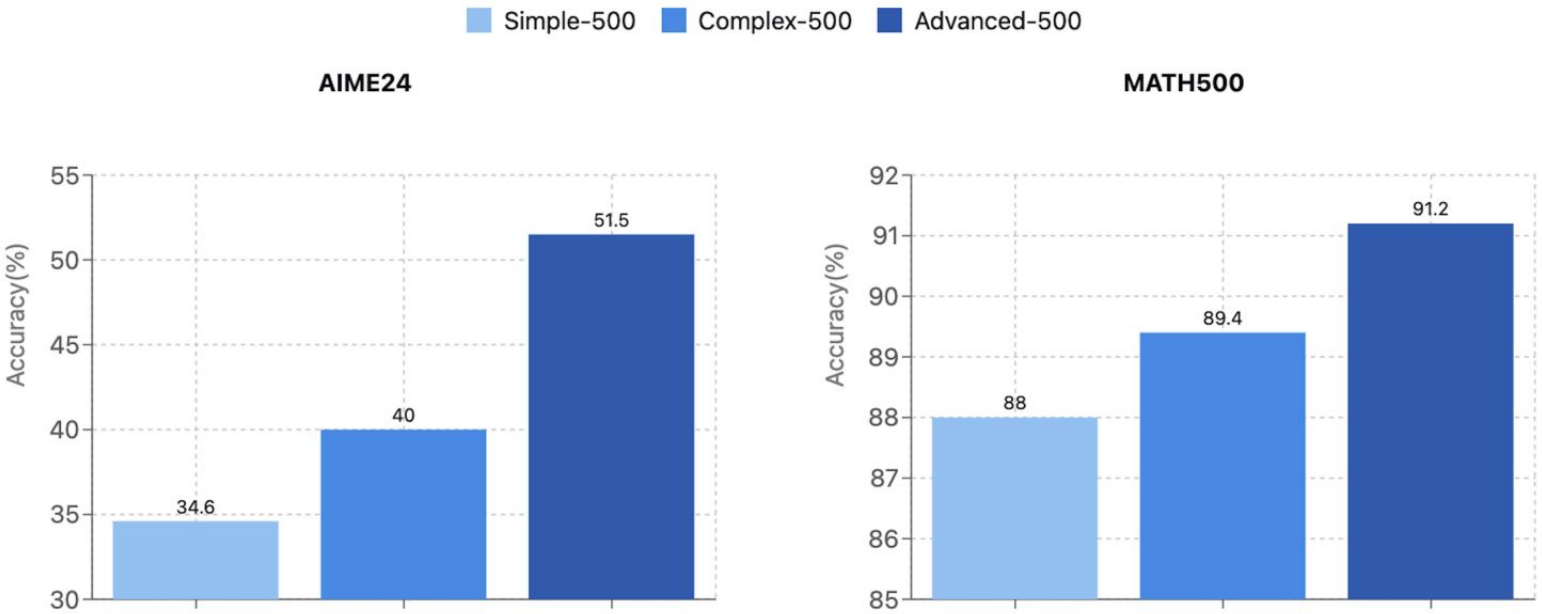
… while excelling across diverse mathematical and multi-discipline benchmarks



LIMO achieves superior performance despite using significantly fewer training examples

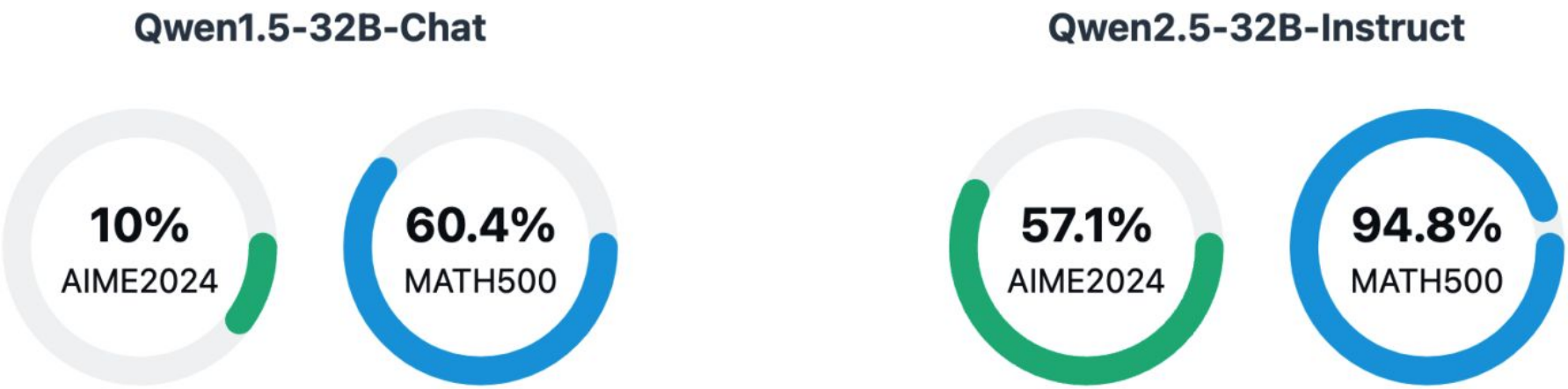
Datasets	OpenAI-o1 -preview	Qwen2.5-32B -Instruct	QwQ-32B-preview	OpenThoughts (114k)	NuminaMath (100k)	LIMO ours(817)
In Domain						
AIME24	44.6	16.5	50.0	50.2	6.5	**57.1**
MATH500	85.5	79.4	89.8	80.6	59.2	**94.8**
AMC23	81.8	64.0	83.6	80.5	40.6	**92.0**
Out of Domain						
OlympiadBench	52.1	45.3	58.5	56.3	36.7	**66.8**
CHMath	50.0	27.3	68.5	74.1	11.2	**75.4**
Gaokao	62.1	72.1	80.1	63.2	49.4	**81.0**
Kaoyan	51.5	48.2	70.3	54.7	32.7	**73.4**
GradeSchool	62.8	56.7	63.8	39.0	36.2	**76.2**
Minerva	**47.1**	41.2	39.0	41.1	24.6	44.9
GPQA	**73.3**	48.0	65.1	42.9	25.8	66.7
AVG.	61.1	49.9	66.9	58.3	32.3	**72.8**

Models trained on different question quality



- **Simple-500**: 500 simple problems randomly selected problems from MATH levels 1 and 2.
- **Complex-500**: 500 complex problems randomly selected problems from MATH levels 3, 4, and 5.
- **Advanced-500**: 500 advanced problems randomly selected problems from past AIME tests.

Impact of pretrained model choice



Example model outputs

Problem: Every morning Aya goes for a 9 km long walk and stops at a coffee shop afterwards. When she walks at a constant speed of s kilometers per hour, the walk takes her 4 hours, including t minutes spent in the coffee shop. When she walks $s+2$ kilometers per hour, the walk takes her 2 hours and 24 minutes, including t minutes spent in the coffee shop. Suppose Aya walks at $s+\\frac{1}{2}$ kilometers per hour. Find the number of minutes the walk takes her, including the t minutes spent in the coffee shop.

Qwen2.5-32B-Instruct

To solve the problem, we need to determine the time Aya spends walking [omitted for brevity…]
The time spent walking is the distance divided by the speed, so the time spent walking is (9/s) hours. Therefore, the equation for the total time is: **9/s + t = 240,** where t is the time spent in the coffee shop in minutes.
Next, we know that [omitted for brevity…]. The time spent walking is 9/(s+2) hours. Therefore, the equation for the total time is: **9/(s+2) + t = 144**.
We now have two equations:
1. 9/s + t = 240
2. 9/(s+2) + t = 144
[omitted for brevity…]
So, s = 3 / 16 km/h. Now we substitute s = 3 / 16 km/h back into equation (1) to find t:
9 / (3 / 16) + t = 240, 48 + t = 240, t = 192
[omitted for brevity…]
So, the number of minutes the walk takes her, including the t minutes spent in the coffee shop, is 205.

unable to correct false statements above

Did not cross verify equation 2

DeepSeek-R1

Okay, let's try to figure out this problem step by step. So, Aya goes for a 9-kilometer walk every morning and then stops at a coffee shop. [omitted for brevity…]
First, let's parse the problem. Let's denote the walking [omitted for brevity…]. Our goal is to find the total time when she walks at s+1/2 km/h. **Hmm**, so we need to figure out the values of s and t first.
Let me write down the equations based on the given information. [omitted for brevity…]
Now we have two equations:
1) 9/s + t/60 = 4, 2) 9/(s+2) + t/60 = 2.4
We can subtract the second equation from the first to eliminate [omitted for brevity…]
So the left side becomes: [omitted for brevity…]
Multiply both sides by: [omitted for brevity…]
Now, solving this quadratic equation: [omitted for brevity…]. So, Aya's original speed is 2.5 km/h.
Now, let's find t [omitted for brevity…]. **Wait, let me confirm**. **But wait**, the question says [omitted for brevity…]. Therefore, the answer should be 204 minutes.
But let me just check again to make sure I didn't make any mistakes in the quadratic equation solving. Let's verify that s = 2.5 is correct. [omitted for brevity…]. Thus, the answer should be 204.

think with anthropomorphic tone

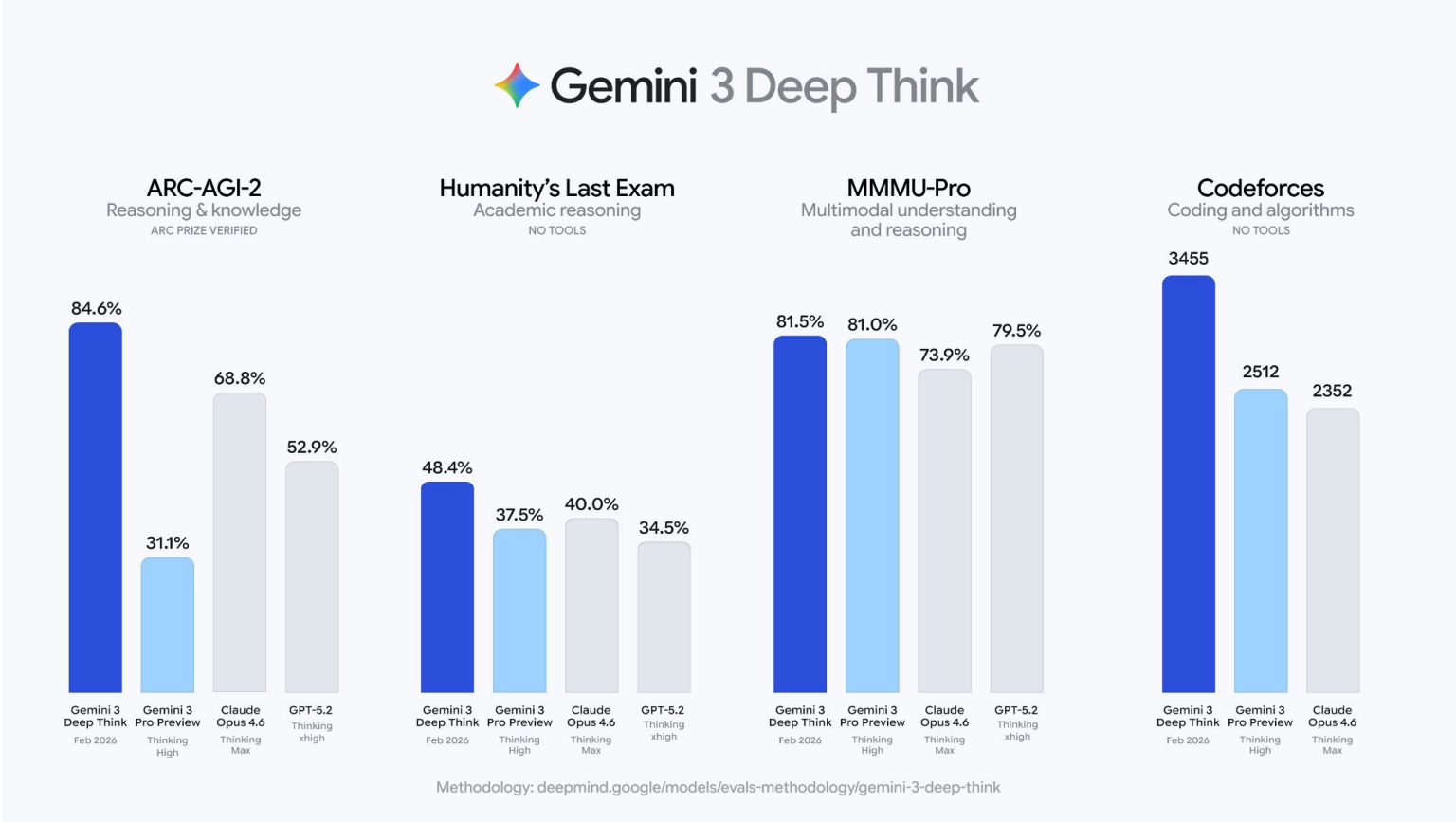
detailed steps in calculation

self-reflection

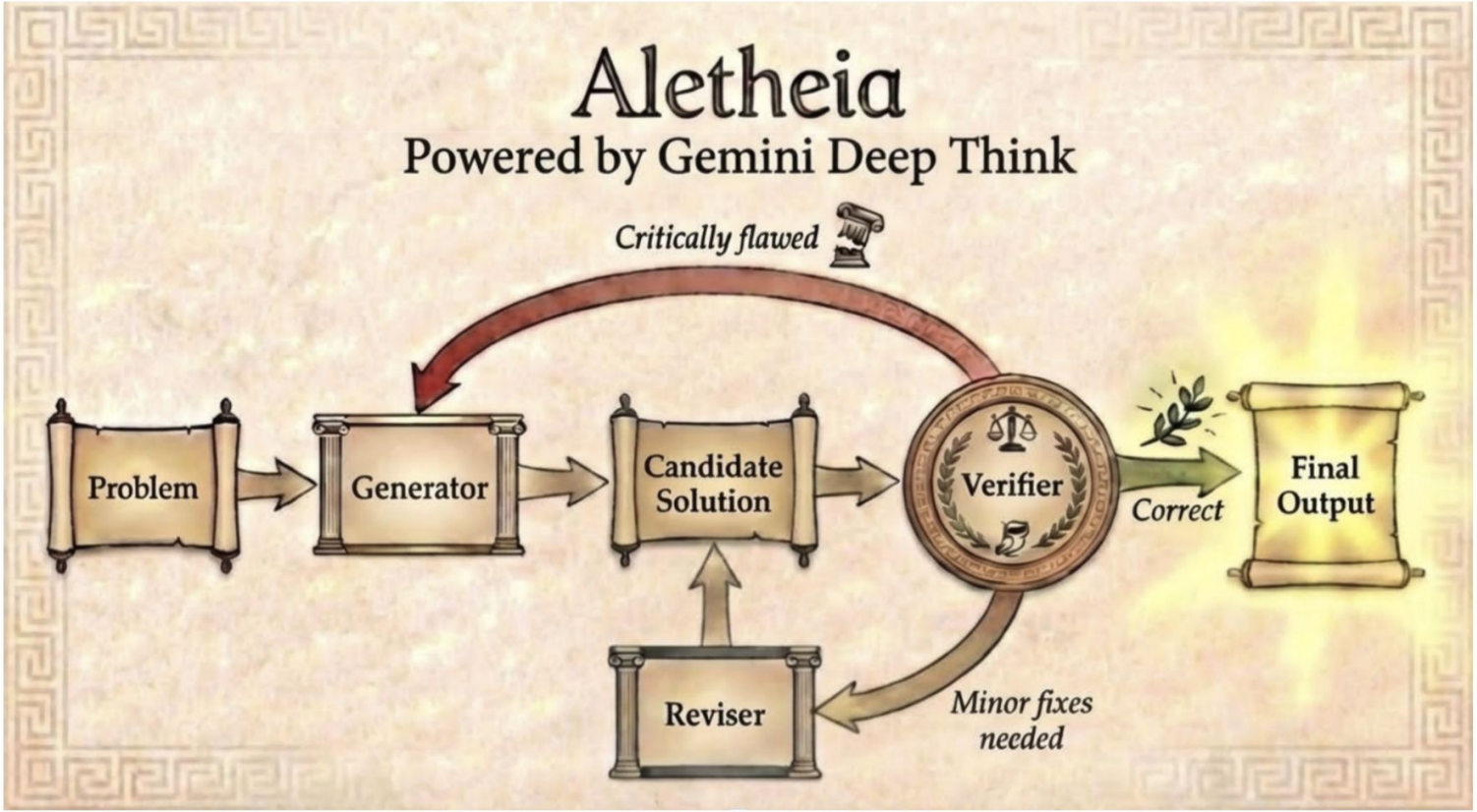
LIMO

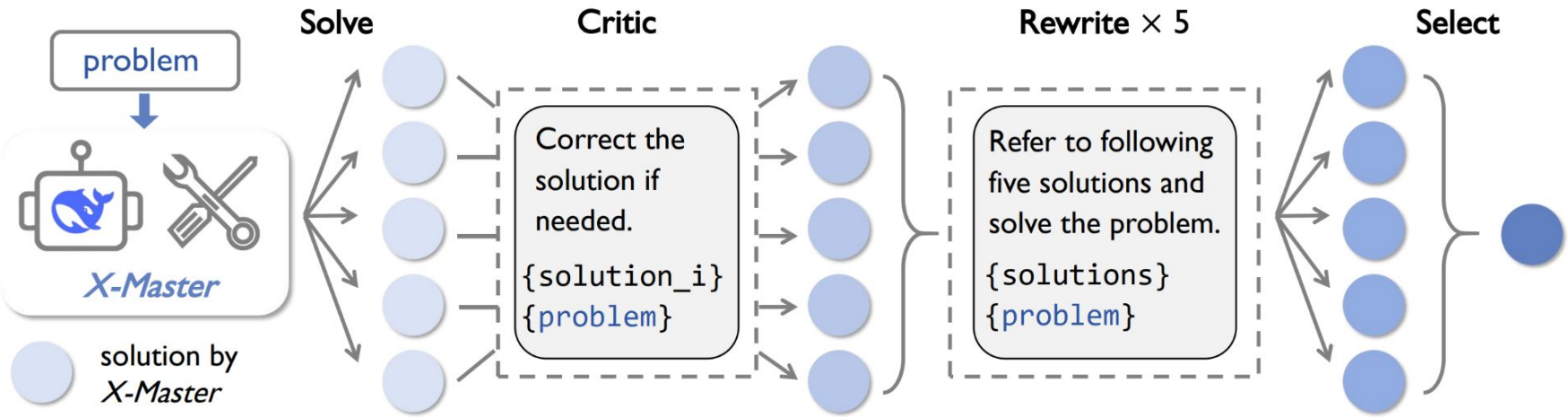
Okay, let's see. So Aya goes for a 9-km walk every morning and then stops at a coffee shop. The time she takes for the walk, including the time in the coffee shop, depends on her walking speed. [omitted for brevity…]. When she walks a bit faster, at s plus 2 kilometers per hour, the total time is 2 hours and 24 minutes, which is 2 hours and 24 minutes, so that's 2.4 hours?
Wait, 24 minutes is 0.4 hours? Wait, no. Wait, 60 minutes is 1 hour, so 24 minutes is 24/60, which is 0.4 hours. So 2 hours and 24 minutes is 2.4 hours. [omitted for brevity…]
So, let's set up equations. Let's denote the walking time as T1 when she walks at s km/h, and T2 when she walks at s+2 km/h. Then, T1 + t = 4 hours, and T2 + t = 2.4 hours. **But t is in minutes, so we need to convert that to hours or convert the total time to minutes. Maybe it's better to keep everything in hours.** So t minutes is t/60 hours. [omitted for brevity…]
For the first case: walking time T1 = 9/s hours, then total time T1 + t/60 = 4 hours.
For the second case: walking time T2 = 9/(s+2) hours, then total time T2 + t/60 = 2.4 hours. So now we have two equations: 1) 9/s + t/60 = 4, 2) 9/(s+2) + t/60 = 2.4
Let me write that as:
9*(1/s - 1/(s+2)) = 1.6. So, 1/s - 1/(s+2) = 1.6/9
Let me compute 1.6 divided by 9. 1.6 divided by 9 is 0.1777... or 16/90, which simplifies to 8/45. So 1.6/9 = 8/45 ≈ 0.1777… So, 1/s - 1/(s+2) = 8/45
Now, let's compute the left side: [omitted for brevity…]. Multiply both sides by 2: [omitted for brevity…]. Let me write that as a quadratic equation: [omitted for brevity…]. Now, let's solve this quadratic equation: [omitted for brevity…]. So, s is 2.5 km/h.
Let's verify that. If s is 2.5 km/h, then walking 9 km would take 9/2.5 = 3.6 hour [omitted for brevity…]
But let me check again. Maybe I made a mistake in calculations. First, s = 2.5 km/h. Then, walking time at s is 9/2.5 = 3.6 hours…[omitted for brevity…] **Alternatively, we could have converted everything to minutes from the start**. Let's see: First walk: 9/(s) hours + t minutes = 4 hours. 4 hours is 240 minutes. [omitted for brevity…] I think that's solid. So the answer is 204.

Gemini Deep Think



Gemini Deep Think





SciMaster: Towards General-Purpose Scientific AI Agents

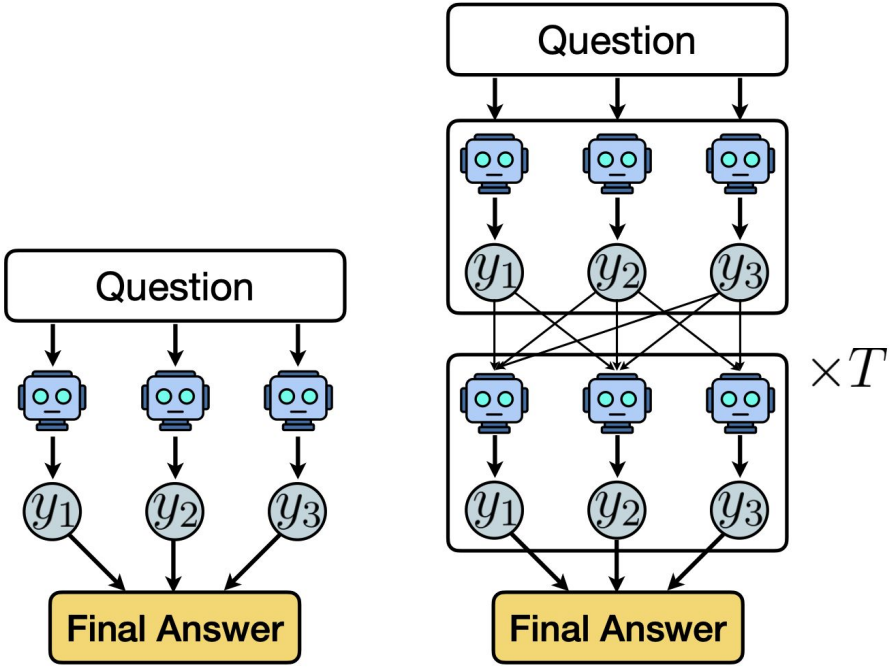
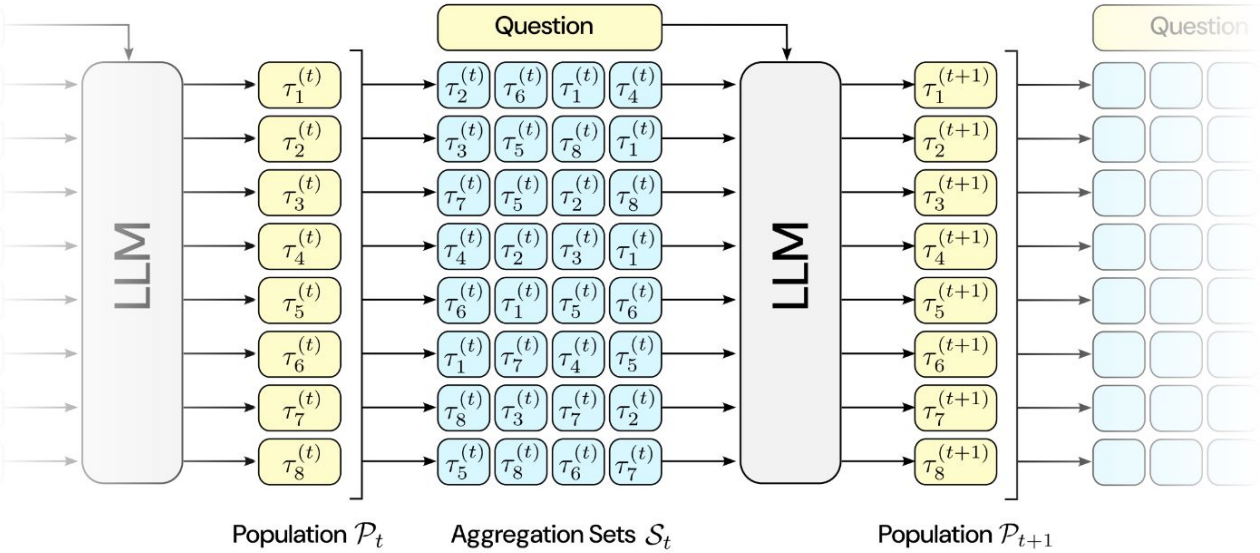


Figure 1: Majority Voting vs. MAD overview.

Debate or Vote: Which Yields Better Decisions in Multi-Agent Large Language Models?

Recursive Self-Aggregation (RSA)



Aggregation Prompt

You are given a problem and several candidate solutions. Some candidates may be incorrect or contain errors. Aggregate the useful ideas and produce a single, high-quality solution.

Reasoning carefully; if candidates disagree, choose the correct path. If all are incorrect, then attempt a different strategy.

End with the final result in <format>.

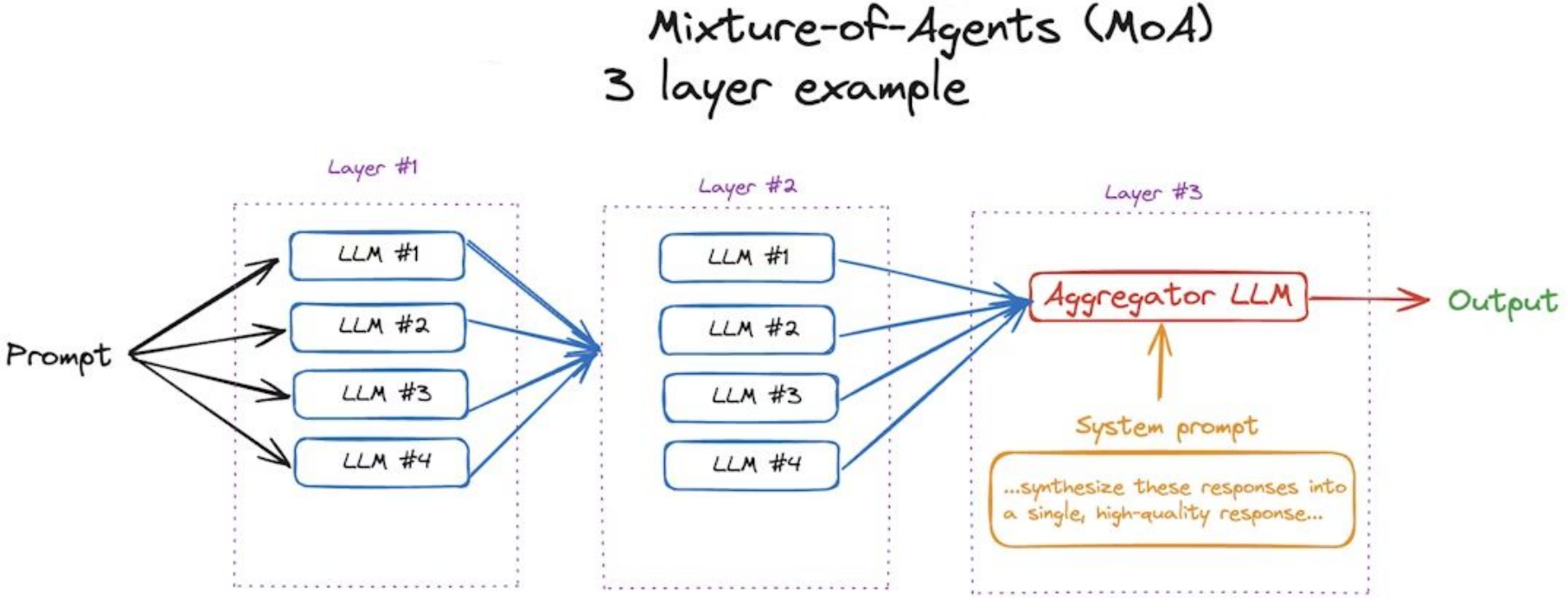
Problem: <problem>

Candidate solutions (may contain mistakes):

---------- Solution 1 ----------
<solution 1>
---------- Solution 2 ----------
<solution 2>
...
---------- Solution K ----------
<solution K>

[Recursive Self-Aggregation Unlocks Deep Thinking in Large Language Models]

Deep Think / Parallel Agents / Mixture-of-Agents



https://docs.together.ai/docs/mixture-of-agents#advanced-moa-example

Thank you!